



Republic of the Philippines
CENTRAL BICOL STATE UNIVERSITY OF AGRICULTURE
Impig, Sipocot, Camarines Sur 4408
Website: www.cbsua.edu.ph
Email Address: cbsua.sipocot@cbsua.edu.ph
Trunkline: (054) 881-6681



Unit 1

Web Programming

Basics

Lesson 4: CSS

Outline

- **CSS**

- How to use CSS?
- Definition

- **Box Model**

- Content and Sizing
- Areas of the Box Model
- Control the Box Model

- **Selector**

- Simple Selector
- Pseudo-Selector

- **Inheritance**

- **Color**

- Named Color
- Numeric Color

- **Sizing Units**

- Percentages
- Dimensions and Lengths

- **Flexbox**

- Main Axis and Cross Axis
- Flex-direction Property
- Wrapping Flex Items

- **Grid**

- Grid Lines
- Grid Tracks
- Grid Cell
- Grid Area
- Grid Gap

- **Positioning**

- Static
- Relative
- Absolute
- Fixed
- Sticky

- **Z-Index**

- Negative z-index
- Stacking Context

- **Media Queries**

- Media Types
- Media Features

Cascading Style Sheets (css)

Cascading Style Sheets (css)

Cascading Style Sheets (CSS)

- A stylesheet language used to describe the **presentation** of a document written in HTML

Cascading Style Sheets (CSS)

- A stylesheet language used to describe the presentation of a document written in HTML

How to use CSS?

Inline

use tag's style attribute to specify appearance

Internal

create <style> elements in HTML and assign to different selectors

External

styling in separate a CSS file

CSS

Inline CSS

```
<body>  
  <h1 style="color: red">My Website</h1>  
</body>
```

CSS

Inline CSS

```
<body>  
  <h1 style="color: red">My Website</h1>  
</body>
```

Internal CSS

```
<head>  
  <style>  
    h1 {  
      color: red;  
      text-transform: capitalize;  
    }  
  </style>  
</head>
```

CSS

Inline CSS

```
<body>
|   <h1 style="color: ■ red">My Website</h1>
|
|   </body>
```

Internal CSS

```
<head>
|   <style>
|       |   h1 {
|       |       |   color: ■ red;
|       |       |   text-transform: capitalize;
|       |       }
|   </style>
|
|   </head>
```

External CSS

```
<head>
|   <link rel="stylesheet" href="style.css">
|
|   </head>
```


CSS Syntax

CSS Selector

```
<style>
  h1 {
    color: ■ red;
    text-transform: capitalize;
  }
</style>
```

Properties and Values

CSS

Box Model

CSS

Box Model

- It is essentially a box that wraps around every HTML Element

CSS

Box Model

- It is essentially a box that wraps around every HTML Element
- It consists of **content, padding, borders, and margins**

Box Model

Content and Sizing

CSS

```
p {  
  width: 100px;  
  height: 50px;  
  padding: 20px;  
  border: 1px solid;  
}
```

HTML

```
<p>I am a paragraph of the text that has a few words in it</p>
```

OUTPUT: ?

How **wide** is the content?

Box Model

Content and Sizing

CSS

```
p {  
  width: 100px;  
  height: 50px;  
  padding: 20px;  
  border: 1px solid;  
}
```

HTML

```
<p>I am a paragraph of the text that has a few words in it</p>
```

OUTPUT

I am a
paragraph of
the text that
has a few
words in it

➤ The content is **142px** wide

Box Model

Content and Sizing

Intrinsic Sizing

Extrinsic Sizing

Box Model

Content and Sizing

Intrinsic Sizing

- The element sizes itself based on its **content**

Extrinsic Sizing

- The element's size is controlled by **external** factors

Box Model

Content and Sizing

Intrinsic Sizing

- The element sizes itself based on its **content**

Extrinsic Sizing

- The element's size is controlled by **external** factors

```
.intrinsic {  
  background-color: lightblue;  
  padding: 10px;  
  border: 1px solid black;  
  display: inline-block;  
}  
  
.extrinsic {  
  background-color: lightgreen;  
  padding: 10px;  
  border: 1px solid black;  
  width: 200px;  
}
```

CSS

Box Model

Content and Sizing

Intrinsic Sizing

- The element sizes itself based on its **content**

Extrinsic Sizing

- The element's size is controlled by **external** factors

```
.intrinsic {  
  background-color: lightblue;  
  padding: 10px;  
  border: 1px solid black;  
  display: inline-block;  
}  
  
.extrinsic {  
  background-color: lightgreen;  
  padding: 10px;  
  border: 1px solid black;  
  width: 200px;  
}
```

CSS

```
<h3>Intrinsic (size based on content)</h3>  
<div class="intrinsic">  
  This box sizes itself to fit this text.  
</div>  
<h3>Extrinsic (size based on CSS)</h3>  
<div class="extrinsic">  
  This box has fixed width of 200px.  
</div>
```

HTML

Box Model

Content and Sizing

Intrinsic Sizing

- The element sizes itself based on its **content**

Extrinsic Sizing

- The element's size is controlled by **external** factors

```
.intrinsic {  
  background-color: lightblue;  
  padding: 10px;  
  border: 1px solid black;  
  display: inline-block;  
}  
  
.extrinsic {  
  background-color: lightgreen;  
  padding: 10px;  
  border: 1px solid black;  
  width: 200px;  
}
```

CSS

```
<h3>Intrinsic (size based on content)</h3>  
<div class="intrinsic">  
  This box sizes itself to fit this text.  
</div>  
<h3>Extrinsic (size based on CSS)</h3>  
<div class="extrinsic">  
  This box has fixed width of 200px.  
</div>
```

HTML

Intrinsic (size based on content)

This box sizes itself to fit this text.

Extrinsic (size based on CSS)

This box has fixed width of 200px.

OUTPUT

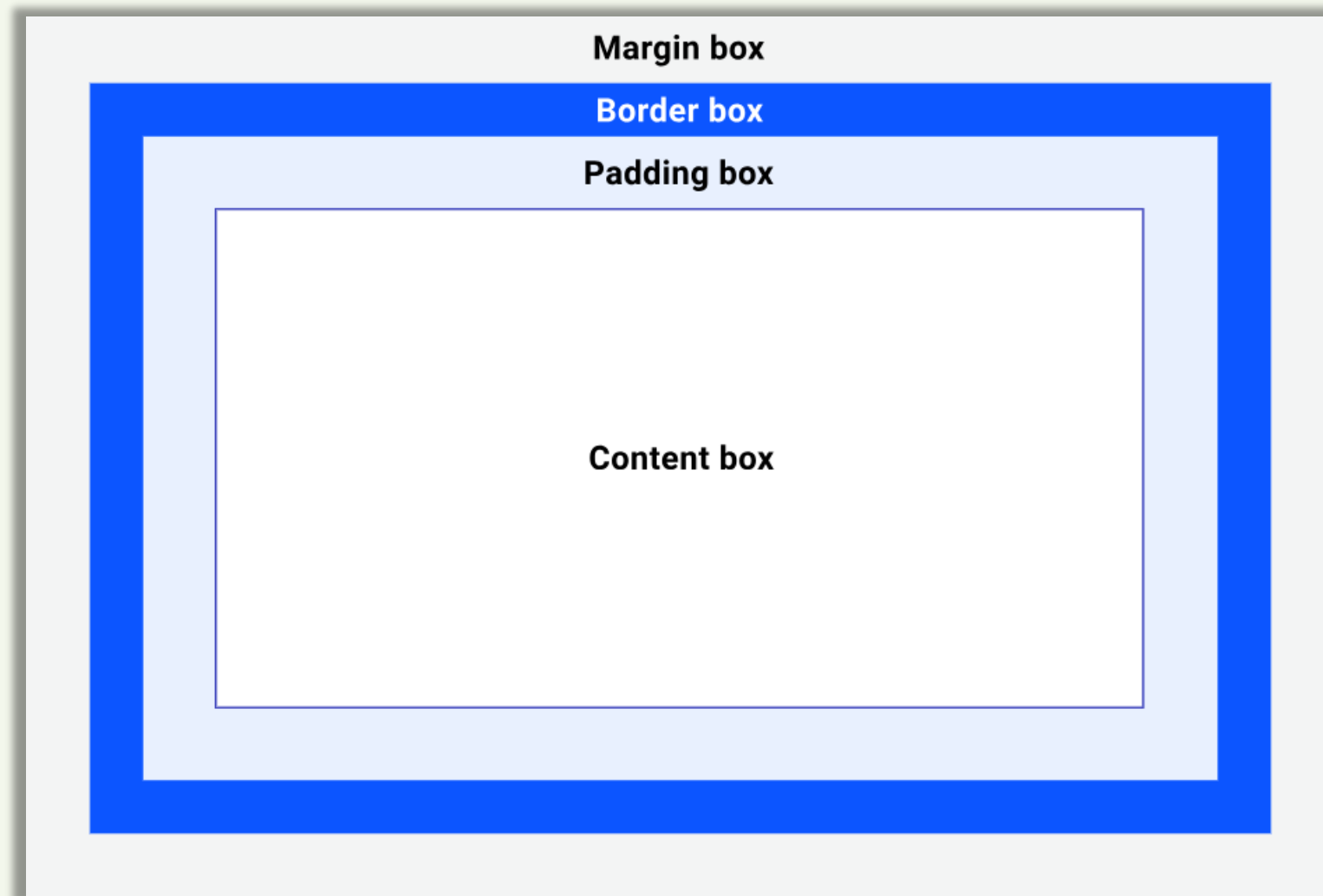
Box Model

Content and Sizing

- Boxes have different behavior based on their **display** value, their set **dimensions**, and the **content** they contain.
- Intrinsic sizing is the browser's default behavior

Box Model

Areas of the Box Model



Box Model

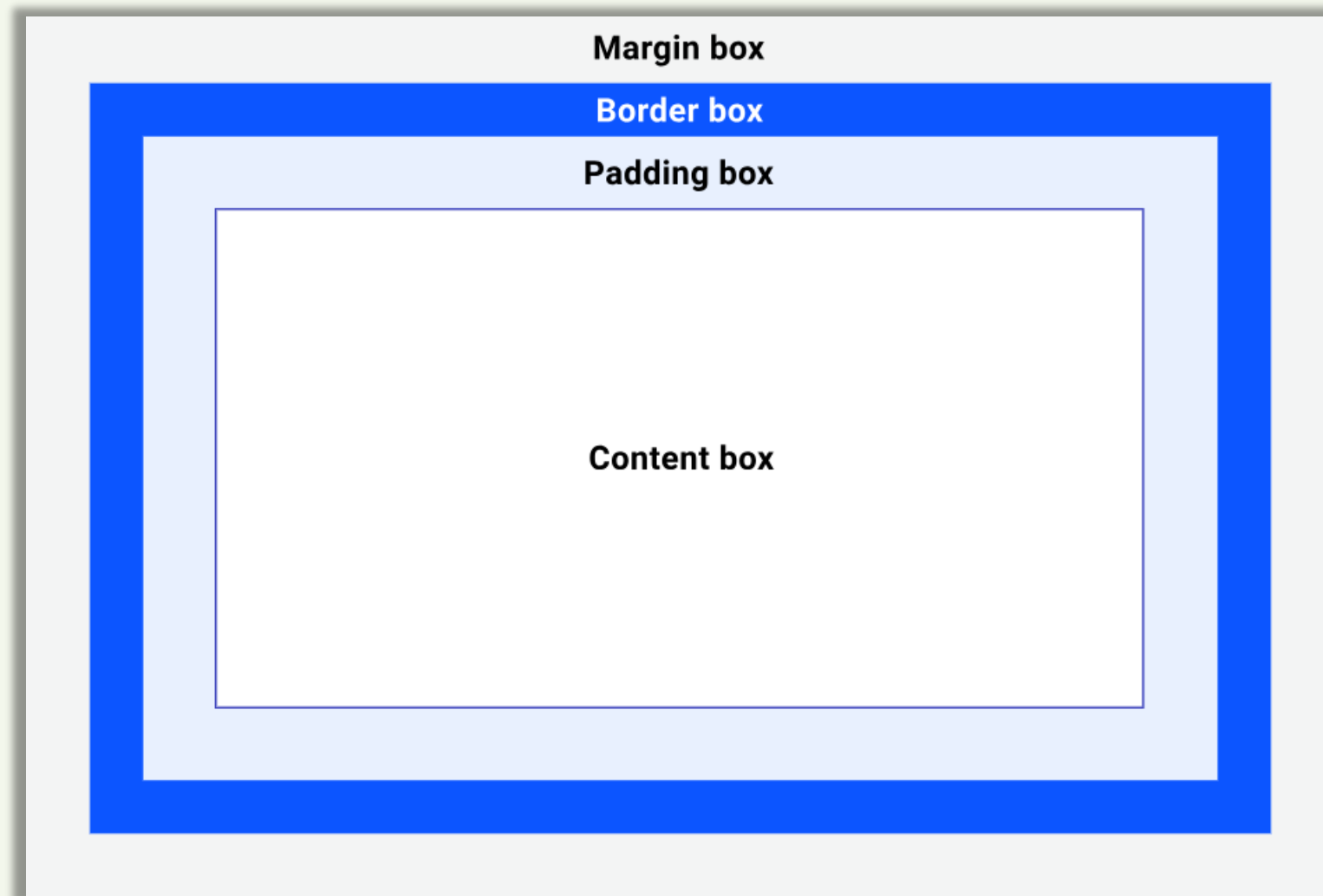
Areas of the Box Model

Content Box

- it is the area that the content lives in. The content can control the size of its parent

Box Model

Areas of the Box Model



Box Model

Areas of the Box Model

Padding Box

- surrounds the content box and is the space created by the **padding property**

Box Model

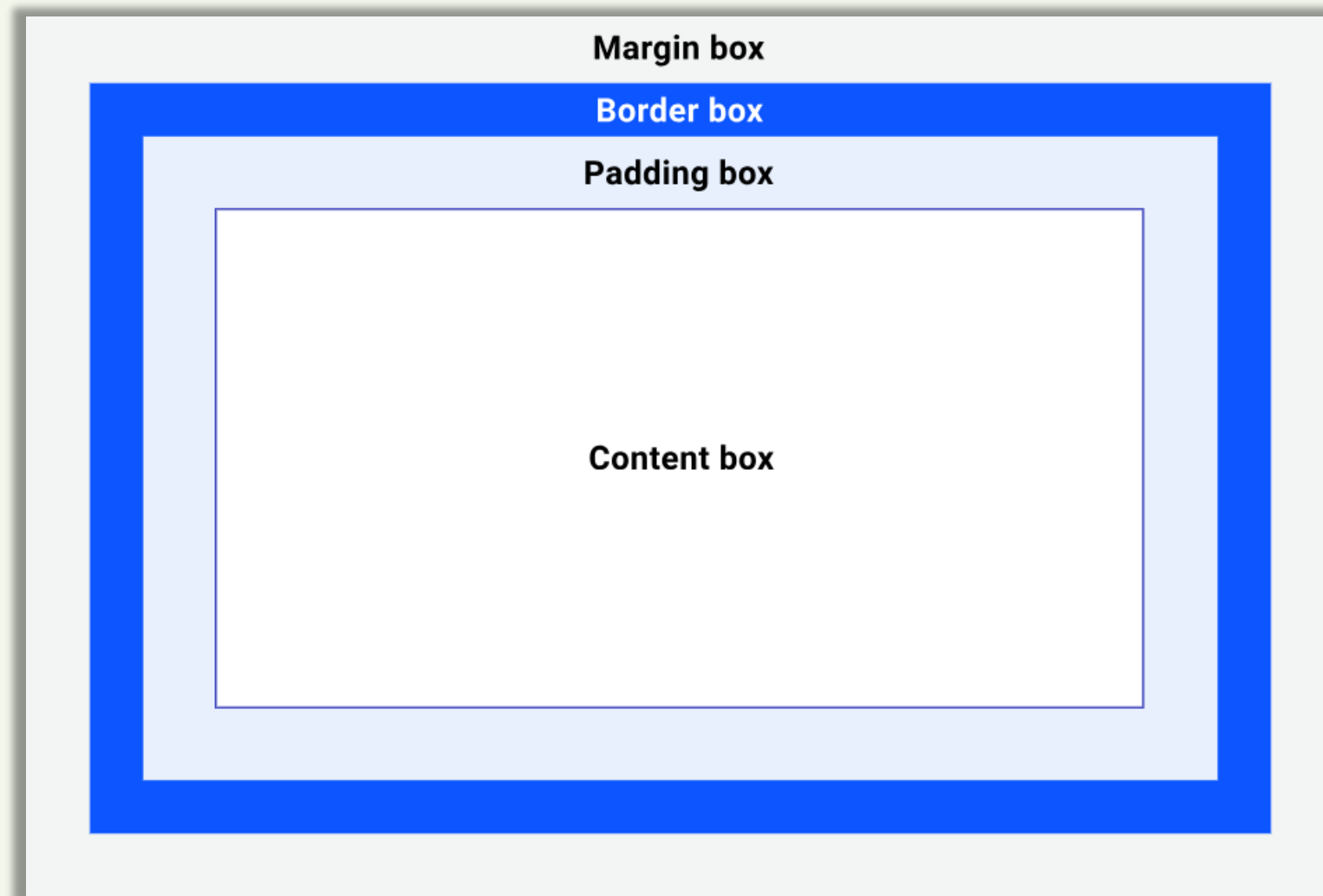
Areas of the Box Model

Padding Box

- surrounds the content box and is the space created by the **padding property**
- since the padding is inside the box, the box's background is **visible** in the space it creates

Box Model

Areas of the Box Model



Box Model

Areas of the Box Model

Border Box

- surrounds the padding box, and its space is defined by the **border value**, which creates a visual frame for the element.

Box Model

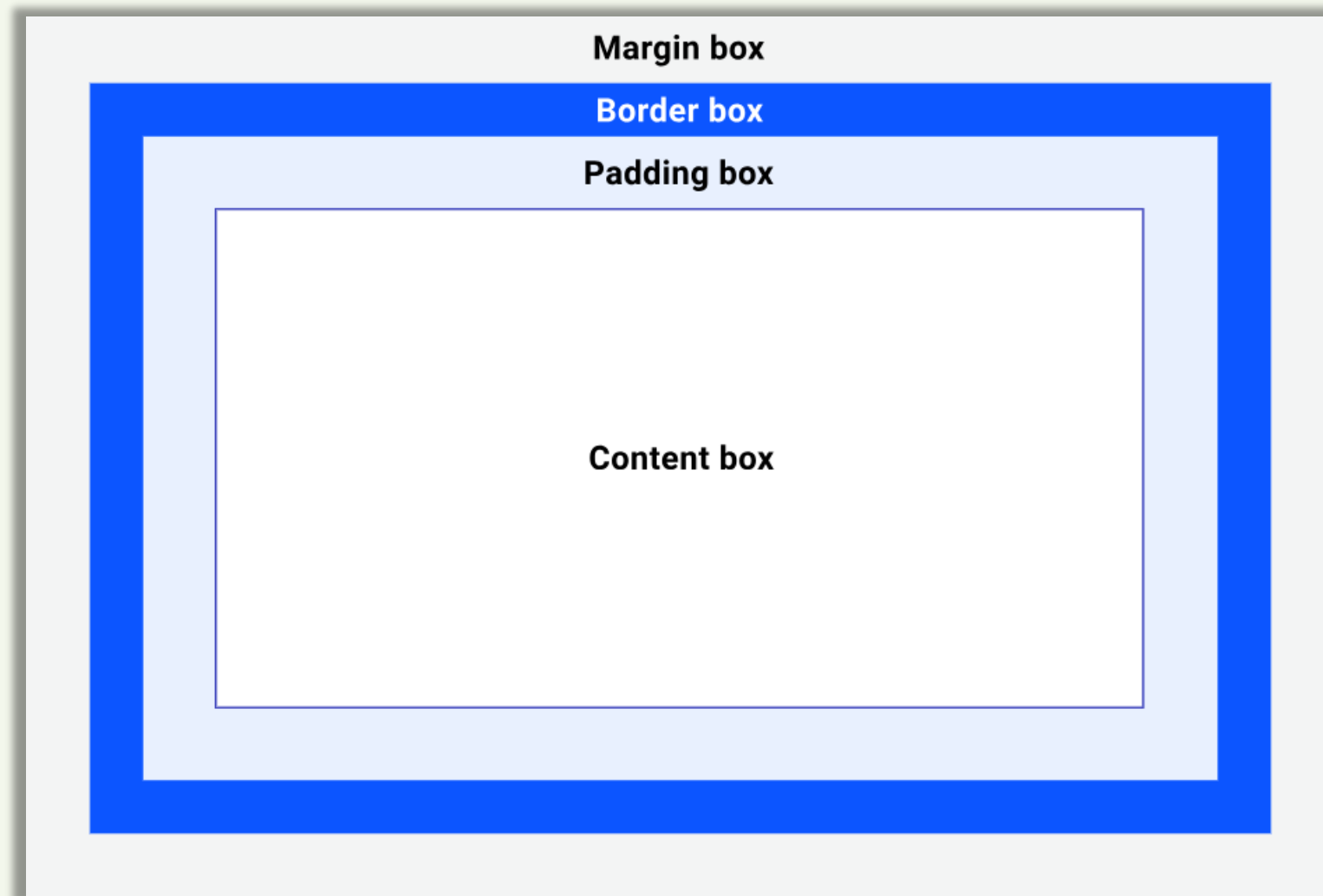
Areas of the Box Model

Border Box

- surrounds the padding box, and its space is defined by the **border value**, which creates a visual frame for the element.
- the element's border edge is the limit of what you can see.

Box Model

Areas of the Box Model



Box Model

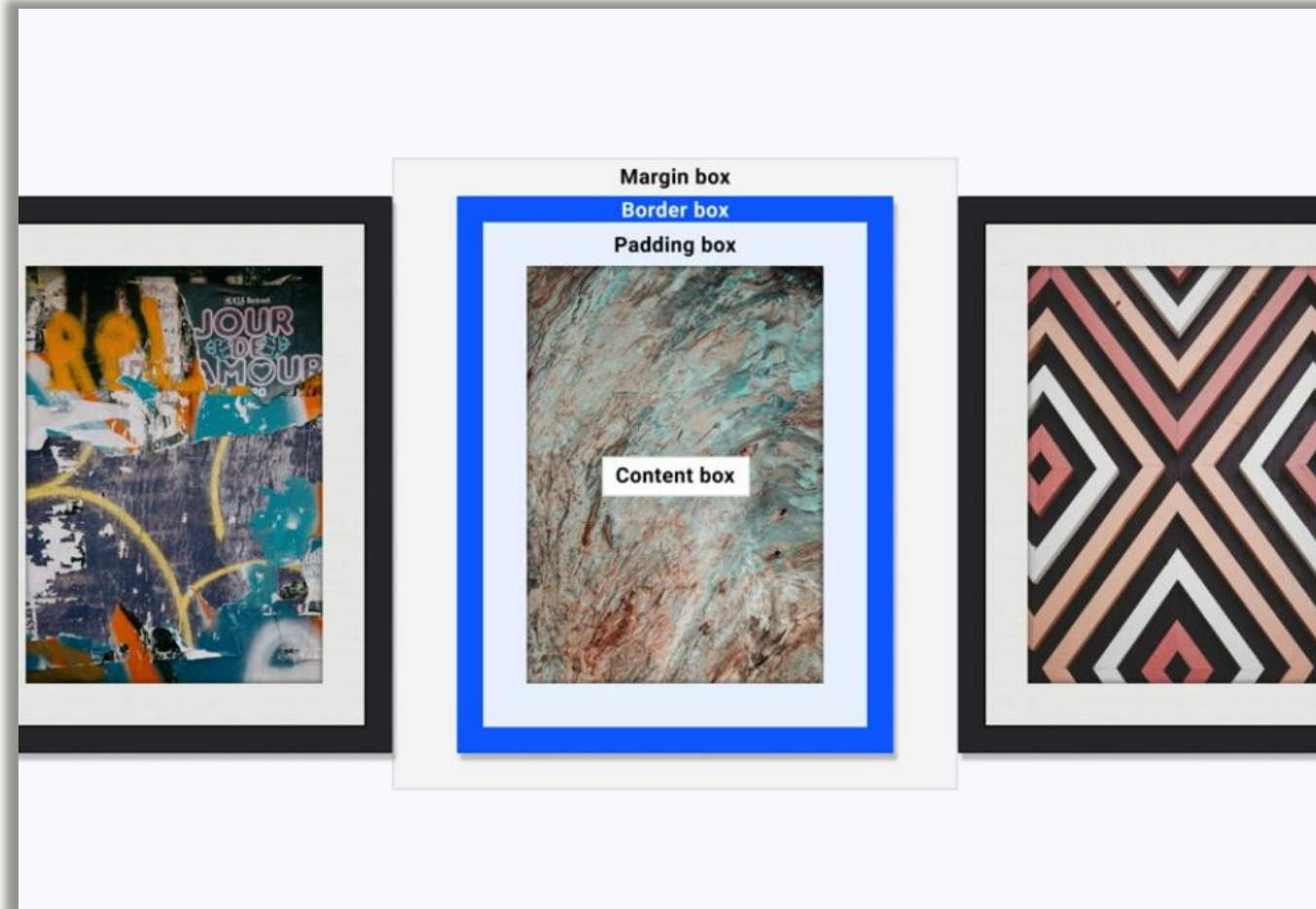
Areas of the Box Model

Margin Box

- it is the space around the element's box, defined by the box's **margin rule**

Box Model

Analogy for Box Model



Box Model

Analogy for Box Model

- ❑ The content box is the **artwork**.
- ❑ The padding box is **the white mounting board**, between the frame and the artwork.
- ❑ The border box is the **frame**, providing a literal border for the artwork.
- ❑ The margin box is the **space between** frames.
- ❑ The **shadow** occupies the same space as the margin box.

Box Model

Areas of the Box Model

Example

```
<style>
  .box {
    width: 200px;
    height: 100px;
    padding: 20px;
    border: 5px solid blue;
    margin: 15px;
    background-color: lightyellow;
  }
</style>
```

```
<h2>CSS Box Model Demo</h2>
<div class="box">
  Content goes here
</div>
```

CSS Box Model Demo

Content goes here

Box Model

Control the Box Model

Box Model

Control the Box Model

- One property the user **agent style sheet** sets a box's default *display*

Box Model

Control the Box Model

- One property the user **agent style sheet** sets a box's default *display*

Example

- The <div> element's default display value is block.
- The element's default display value is list-item.
- The element's default display value is inline.

Box Model

Control the Box Model

- ***inline*** elements have block margins, but other elements ignore these margins.
- ***inline-block*** elements keep inline behavior but other elements recognize their block margins.
- ***block*** elements fill the available horizontal space by default, while inline and inline-block elements size only to their content.

Box Model

Control the Box Model

- The user agent style sheet sets a default ***box-sizing: content-box;*** for all elements.
- This means width and height apply only to the content area. ***padding*** and ***border*** are added outside this size.

Box Model

Control the Box Model

```
<style>
  .inline {
    display: inline;
    margin: 20px;
    padding: 10px;
    border: 2px solid red;
    background-color: lightcoral;
  }

  .inline-block {
    display: inline-block;
    margin: 20px;
    padding: 10px;
    border: 2px solid green;
    background-color: lightgreen;
  }

  .block {
    display: block;
    margin: 20px;
    padding: 10px;
    border: 2px solid blue;
    background-color: lightblue;
  }
</style>
```

```
<body>
  <h2>Box Model & Display Types</h2>

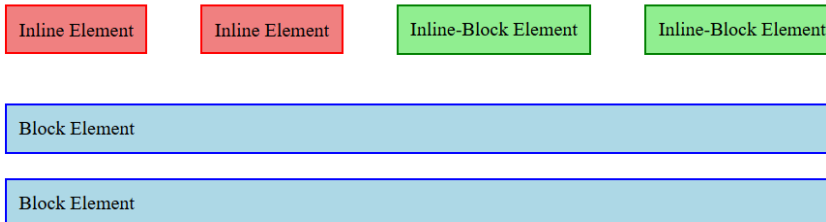
  <span class="inline">Inline Element</span>
  <span class="inline">Inline Element</span>

  <div class="inline-block">Inline-Block Element</div>
  <div class="inline-block">Inline-Block Element</div>

  <div class="block">Block Element</div>
  <div class="block">Block Element</div>

</body>
```

Box Model & Display Types



CSS Selector

CSS Selector

CSS provides options to select elements and apply rules to them

Selector

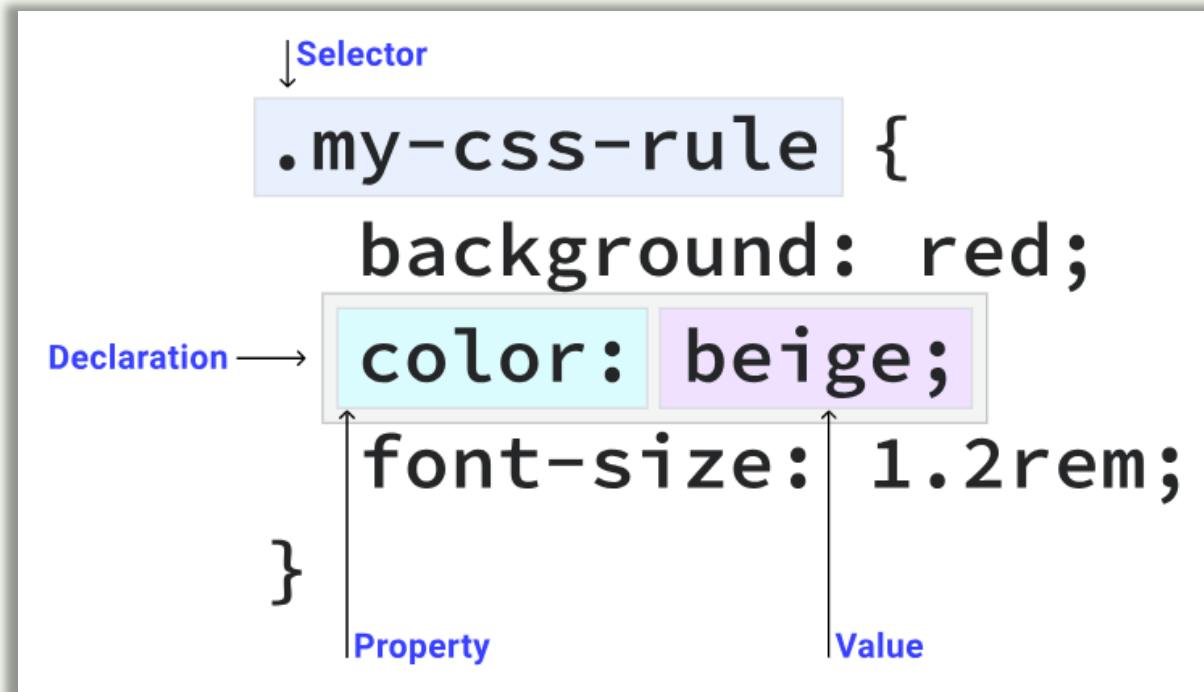
Parts of a CSS rule

- A CSS rule is a block of code, containing one or more selectors and one or more declarations

Selector

Parts of a CSS rule

- A CSS rule is a block of code, containing one or more selectors and one or more declarations



Selector

Simple Selectors

Selector

Simple Selectors

Universal Selector

- (a.k.a. wildcard) matches any element

Syntax

```
* {  
  /* CSS properties */  
}
```

Example

```
* {  
  color:  hotpink;  
}
```

Selector

Simple Selectors

Universal Selector

- (a.k.a. wildcard) matches any element

Syntax

```
* {  
  /* CSS properties */  
}
```

Example

```
* {  
  color:  hotpink;  
}
```

This rule causes every HTML element on the page to have *hotpink* text.

Selector

Simple Selectors

Type Selector

- matches an HTML element directly

Syntax

```
element {  
    /* CSS properties */  
}
```

Example

```
p {  
    border: 1px solid;  
}
```

Selector

Simple Selectors

Class Selector

- matches any element that has that class applied to it

Syntax

```
.className {  
    /* CSS properties */  
}
```

Example

```
.my-class {  
    color:  red;  
}
```


Selector

Simple Selectors

ID Selector

- Target an element with a specific id attribute

Syntax

```
#idName {  
    /* CSS properties */  
}
```

Example

```
#rad {  
    border: 1px solid  blue;  
}
```

Selector

Simple Selectors

Attribute Selector

- targets elements based on the presence or value of their attributes

Syntax

```
[element] {  
    /* CSS properties */  
}
```

Example

```
input[type="text"] {  
    border: 1px solid black;  
}
```

Selector

Simple Selectors

Grouping Selector

- Apply the same styles to multiple elements at once

Syntax

```
selector1, selector2, selector3 {  
    /* CSS properties */  
}
```

Example

```
h1,  
h2,  
p {  
    color:  darkblue;  
}
```

Selector

Pseudo Selectors

Pseudo-classes

- apply styles based on state changes and external factors

Syntax

```
selector:pseudo-class {  
    /* CSS properties */  
}
```

Example

```
a:hover {  
    color:  red;  
}
```

Selector

Pseudo Selectors

Pseudo-classes

Pseudo-Class	Description
:active	Element being clicked
:any-link	All links (visited or not)
:checked	Checked checkboxes/radio buttons
:disabled	Disabled form elements
:empty	Elements with no content
:enabled	Enabled form elements
:first-child	First child of its parent
:first-of-type	First of its type within parent
:focus	Element currently focused
:hover	Element when mouse is over it
:in-range	Input elements with value in valid range
:invalid	Input elements with invalid value
:lang()	Elements with a specific language
:last-child	Last child of its parent

Selector

Pseudo Selectors

Pseudo-classes

:last-of-type	Last of its type within parent
:link	Unvisited links
:not()	Excludes elements matching selector
:nth-child()	Element at a specific position
:nth-of-type()	Element of a type at a specific position
:only-child	Only child of its parent
:only-of-type	Only element of its type within parent
:optional	Form inputs not required
:placeholder-shown	Inputs showing placeholder text
:read-only	Non-editable elements
:required	Form inputs marked as required
:root	Document root element (usually <html>)
:target	Element targeted by URL fragment
:valid	Form inputs with valid value
:visited	Links already visited
:where()	Selector with zero specificity

Selector

Pseudo Selectors

Pseudo-element

- adding or targeting an extra element without having to add more HTML

Syntax

```
selector::pseudo-element {  
  /* CSS properties */  
}
```

Example

```
p::first-letter {  
  font-size: 2em;  
}
```

Selector

Pseudo Selectors

Pseudo-Element

Pseudo-Element	Description
::after	Inserts content after an element's content.
::before	Inserts content before an element's content.
::placeholder	Styles placeholder text in input and textarea.
::selection	Styles text selected by the user.
::first-letter	Styles the first letter of a block element.
::first-line	Styles the first line of a block element.
::marker	Styles list-item markers like bullets or numbers.

CSS

Inheritance

CSS

Inheritance

CSS

Inheritance

Inherited by Default

Properties related to text and appearance usually inherit.
Example: color, font-family, line-height

Not Inherited by Default

Box model, layout, sizing, and most visual properties do not inherit.
Example: margin, padding, border, width, height

CSS

Inheritance

Force Inheritance

```
property: inherit;
```

make any property inherit

```
* {  
  all: inherit;  
}
```

make all property inherit

CSS

Inheritance

Inheritance is from Parent to Child

Example

```
<div style="color: red;">  
  <p>This text is red.</p>  
</div>
```

Output

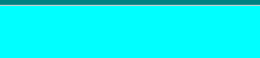
This text is red.

- ❑ `<p>` inherits the color from the parent `<div>`.

Color

Color

Named Color

Keyword	RGB hex value	Sample
black	#000000	
silver	#c0c0c0	
gray	#808080	
white	#ffffff	
maroon	#800000	
red	#ff0000	
purple	#800080	
fuchsia	#ff00ff	
green	#008000	
lime	#00ff00	
olive	#808000	
yellow	#ffff00	
navy	#000080	
blue	#0000ff	
teal	#008080	
aqua	#00ffff	

Syntax

```
color: red;  
color: orange;  
color: tan;  
color: rebeccapurple;  
color: transparent;
```

Color

Numeric Color

HEX Colors

The hexadecimal ranges are **0–9** and **A–F**.

Color

Numeric Color

HEX Colors

The hexadecimal ranges are **0–9** and **A–F**.

When used in a six-digit sequence, they are translated to the RGB numerical ranges which are 0–255 which correspond to the red, green, and blue color channels respectively.

Color

Numeric Color

HEX Colors

The hexadecimal ranges are **0–9** and **A–F**.

When used in a six-digit sequence, they are translated to the RGB numerical ranges which are 0–255 which correspond to the red, green, and blue color channels respectively.

Example

```
h1 {  
  color: #b71540;  
}
```

Color

Numeric Color

Example

```
<style>
  h1 {
    color:  #B71540;
  }
</style>
</head>

<body>
  <h1>Hello World</h1>
</body>
```

Hello World

Color

Numeric Color

HEX Code with Alpha

- Alpha value sets transparency for colors.
- In hex, add two extra digits for alpha.

Example

Black: **#000000**

Black with 50% transparency: **#00000080**

Color

Numeric Color

Example

```
<style>
  h1 {
    color: #00000080;
  }
</style>
</head>

<body>
  <h1>Hello World</h1>
</body>
```

Hello World

Color

Numeric Color

RGB (Red, Green, Blue) Colors

- RGB colors use **rgb()** with numbers (0–255) or percentages (0%–100%)

Example

```
h1 {  
  color: rgb(183 21 64);  
}
```

- ❑ Black: `rgb(0 0 0)` or `rgb(0% 0% 0%)`
- ❑ White: `rgb(255 255 255)` or `rgb(100% 100% 100%)`

Color

Numeric Color

Example

```
<style>
  h1 {
    color:  rgb(183, 21, 64);
  }
</style>
</head>

<body>
  <h1>Hello World</h1>
</body>
```

Hello World

Color

Numeric Color

RGB (Red, Green, Blue) Colors

- Add / after RGB values or use the rgba() function.
- Alpha is set with a percentage or a decimal between 0 and 1.

Example

❑ `rgb(0 0 0 / 50%)`

❑ `rgba(0, 0, 0, 0.5)`

Color

Numeric Color

Example

```
<style>
  h1 {
    color:  rgb(183 21 64 / 50%);
  }
</style>
</head>

<body>
  <h1>Hello World</h1>
</body>
```

Hello World

Color

Numeric Color

HSL (Hue, Saturation, Lightness)

Syntax

hsl(hue saturation lightness) or hsl(angle saturation lightness)

Example

```
h1 {  
  color: hsl(344 79% 40%);  
}
```

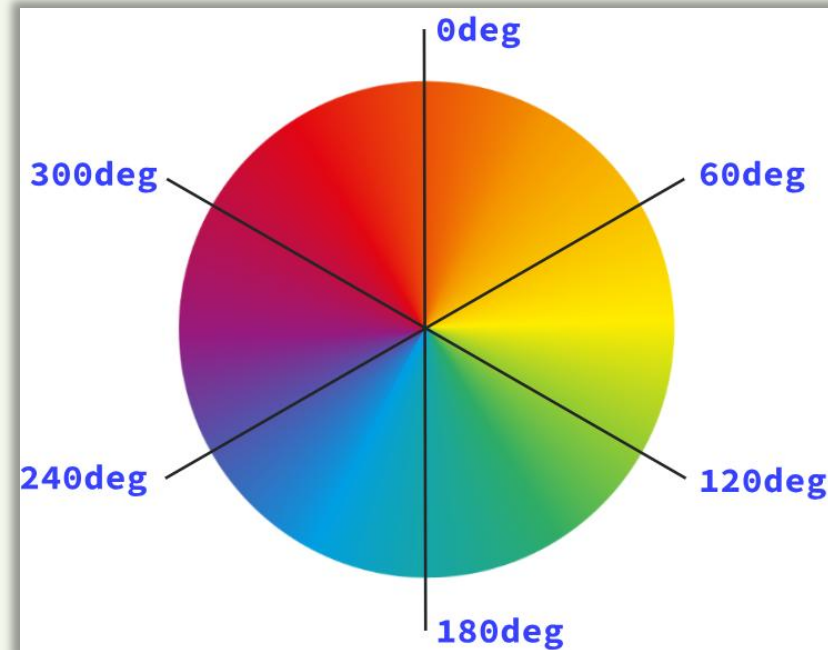
Color

Numeric Color

HSL (Hue, Saturation, Lightness)

Hue

Color position on the color wheel (0–360°); 0°/360° = red, 180° ≈ blue



Color

Numeric Color

HSL (Hue, Saturation, Lightness)

Saturation

Color intensity; 0% = grayscale

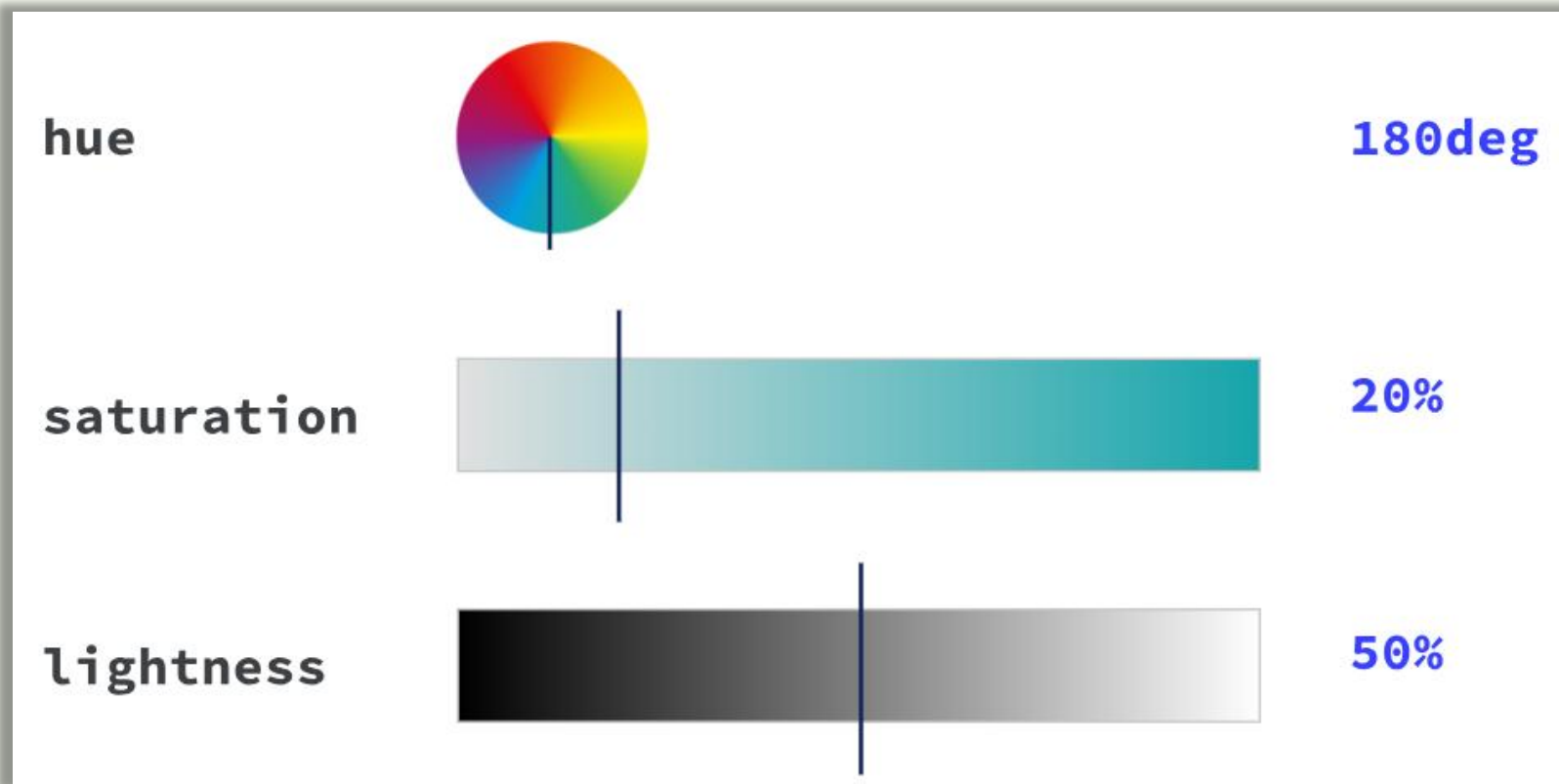
Lightness

Brightness; 100% = white, 0% = black

Color

Numeric Color

HSL (Hue, Saturation, Lightness)



Color

Numeric Color

Example

```
<style>
  h1 {
    color:  hsl(344 79% 40%);
  }
</style>
</head>

<body>
  <h1>Hello World</h1>
</body>
```

Hello World

CSS

Sizing Units

Sizing Units

Percentages

A percentage unit is relative to the parent element's dimensions

```
div {  
  width: 300px;  
  height: 100px;  
}  
  
div p {  
  width: 50%;  
}
```

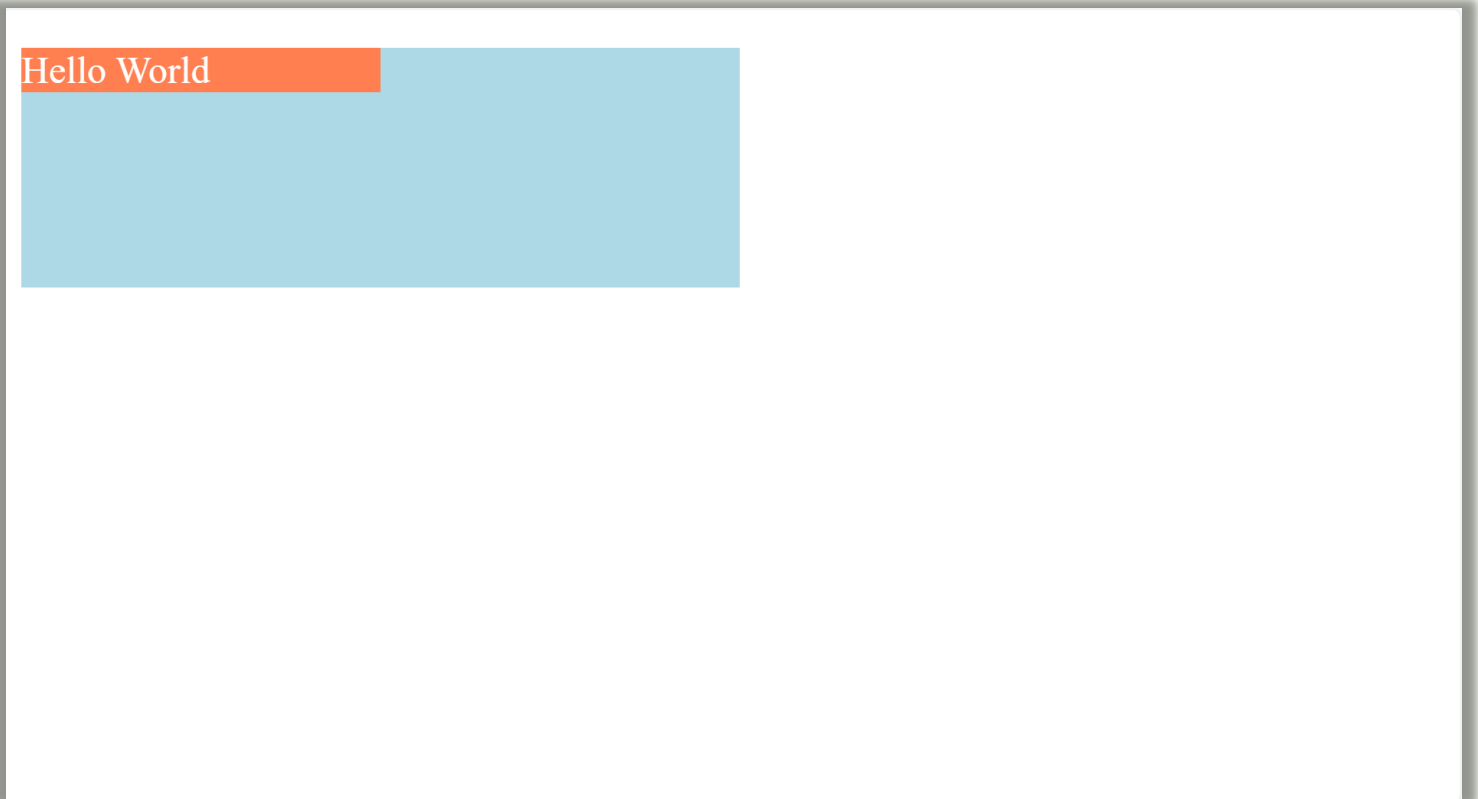

Sizing Units

Percentages

```
<style>
  div {
    width: 300px;
    height: 100px;
    background-color: lightblue;
  }

  p {
    width: 50%;
    background-color: coral;
    color: white;
  }
</style>
```

```
<body>
  <div>
    <p>Hello World</p>
  </div>
</body>
```



Sizing Units

Dimensions and Lengths

A **dimension** is a number with a unit

Lengths are dimensions that measure distance, either **absolute** or **relative**

Dimensions and Length

Absolute Length

Dimensions and Length

Absolute Length

Absolute lengths have a **fixed reference** and are consistent in CSS

Dimensions and Length

Absolute Length

Absolute lengths have a **fixed reference** and are consistent in CSS

Physical units (e.g., cm, in) are accurate for print, not reliable on screens

Dimensions and Length

Absolute Length

Absolute lengths have a **fixed reference** and are consistent in CSS

Physical units (e.g., cm, in) are accurate for print, not reliable on screens

Screen displays vary in pixel size, causing physical units to display **inconsistently**

Dimensions and Length

Absolute Length

Absolute lengths have a **fixed reference** and are consistent in CSS

Physical units (e.g., cm, in) are accurate for print, not reliable on screens

Screen displays vary in pixel size, causing physical units to display **inconsistently**

Best use: Physical units should be used in print style sheets for accuracy

Dimensions and Length

Absolute Length

```
div {  
    width: 10cm;  
    height: 5cm;  
    background: black;  
}
```


Dimensions and Length

Absolute Length

```
<style>
  div {
    width: 10cm;
    height: 5cm;
    background: ☐ black;
  }
</style>
</head>

<body>

  <div></div>
```



Dimensions and Length

Absolute Length

Unit	Name	Equivalent to
<u>cm</u>	Centimeters	1cm = 37.8px
<u>mm</u>	Millimeters	1mm = 1/10th of 1cm
<u>Q</u>	Quarter-millimeters	1Q = 1/40th of 1cm
<u>in</u>	Inches	1in = 96px
<u>pc</u>	Picas	1pc = 1/6th of 1in
<u>pt</u>	Points	1pt = 1/72th of 1in
<u>px</u>	Pixels	1px = 1/96th of 1in

Dimensions and Length

Relative Length

Dimensions and Length

Relative Length

calculated against a base value

Dimensions and Length

Relative Length

calculated against a base value

can reference various bases like font size or viewport size

- **ch** is based on the font's size
- **vw** is based on the viewport width

Dimensions and Length

Relative Length

Font-Size-Relative Units

CSS offers units relative to typography, like em (text size) and ch (character width)

Dimensions and Length

Relative Length

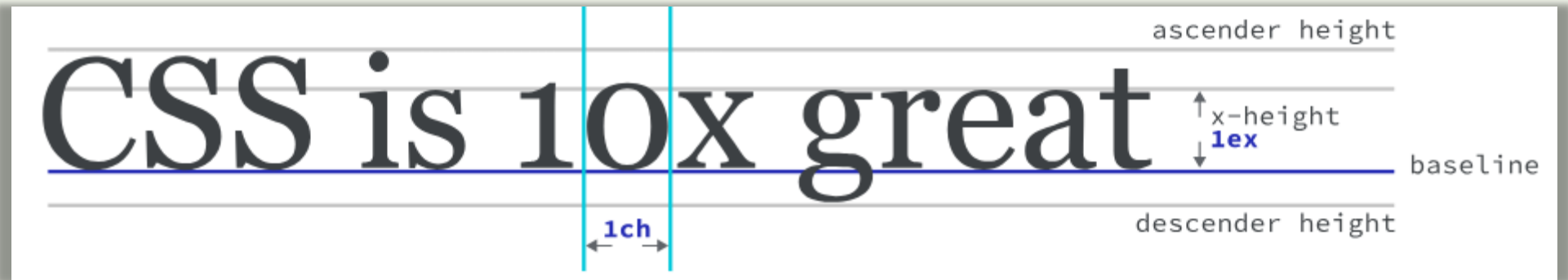
Font-Size-Relative Units

unit	relative to:
<u>em</u>	Relative to the font-size of the element (2em means 2 times the size of the current font)
<u>rem</u>	Font size of the root element (default is 16px)
<u>ex</u>	Relative to the x-height of the current font
<u>rex</u>	The ex value of the root element
<u>cap</u>	Height of the capital letters in the current computed font size of the element
<u>rcap</u>	The cap value of the root element
<u>ch</u>	Relative to the width of the "0" (zero)
<u>rch</u>	The ch value of the root element
<u>lh</u>	Line height of the element
<u>rlh</u>	Line lh value of the root element

Dimensions and Length

Relative Length

Font-Size-Relative Units



Dimensions and Length

Relative Length

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>em Example</title>
  <style>
    body {
      font-size: 16px; /* 1em = 16px */
    }

    p {
      font-size: 1.5em; /* 1.5 x parent font size = 24px */
    }

    .small {
      font-size: 0.5em; /* 0.5 x parent font size = 12px */
    }
  </style>
</head>
<body>
  <p>This is a paragraph with 1.5em font size.</p>
  <p class="small">This is smaller text (0.5em).</p>
</body>
</html>
```

This is a paragraph with 1.5em font size.

This is smaller text (0.5em).

Dimensions and Length

Relative Length

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>rem Example</title>
  <style>
    html {
      font-size: 16px; /* 1rem = 16px */
    }

    body {
      font-size: 1.25rem; /* 1.25 × 16px = 20px */
    }

    p {
      font-size: 2rem; /* 2 × 16px = 32px */
    }

    .small {
      font-size: 0.5rem; /* 0.5 × 16px = 8px */
    }
  </style>
</head>
<body>
  <p>This paragraph uses 2rem font size.</p>
  <p class="small">This is smaller text (0.5rem).</p>
</body>
</html>
```

This paragraph uses 2rem font size.

This is smaller text (0.5rem).

Dimensions and Length

Relative Length

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  font-size: 30px;
  border: 1px solid black;
}

span {
  font-size: 1ex;
}
</style>
</head>
<body>

<div>The font-size of the div element is set to 30px. <span>The
span element inside the div element has a font-size of 1ex</span>.
</div>

</body>
</html>
```

The font-size of the div element is set to 30px. The span element inside the div element has a font-size of 1ex.

Dimensions and Length

Relative Length

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  font-size: 16px;
}

div {
  font-size: 3ch;
  border: 1px solid black;
}
</style>
</head>
<body>

<p>The font-size of this document is 16px.</p>

<div>The font-size of this div element is 3ch.</div>

<p>The ch unit sets the font-size relative to the width of the character "0".</p>

</body>
</html>
```

The font-size of this document is 16px.

The font-size of this div element is 3ch.

The ch unit sets the font-size relative to the width of the character "0".

Dimensions and Length

Relative Length

Viewport-Relative Units

Viewport units divide the available browser window space and can be used as a relative sizing basis

Dimensions and Length

Relative Length

Viewport-Relative Units

unit	relative to
<u>vw</u>	Relative to 1% of the width of the viewport
<u>vh</u>	Relative to 1% of the height of the viewport
<u>vmin</u>	1% of the viewport's smaller dimension
<u>vmax</u>	1% of the viewport's larger dimension

Dimensions and Length

Relative Length

Viewport-Relative Units

```
<style>
  .box1 {
    width: 50vw;
    height: 50vh;
    background-color: lightblue;
    margin-bottom: 20px;
  }

  .box2 {
    width: 50vmax;
    height: 50vmin;
    background-color: lightgreen;
  }
</style>
</head>

<body>
  <div class="box1">vw and vh box</div>
  <div class="box2">vmin and box</div>
</body>
```

Dimensions and Length

Relative Length

Viewport-Relative Units

Example

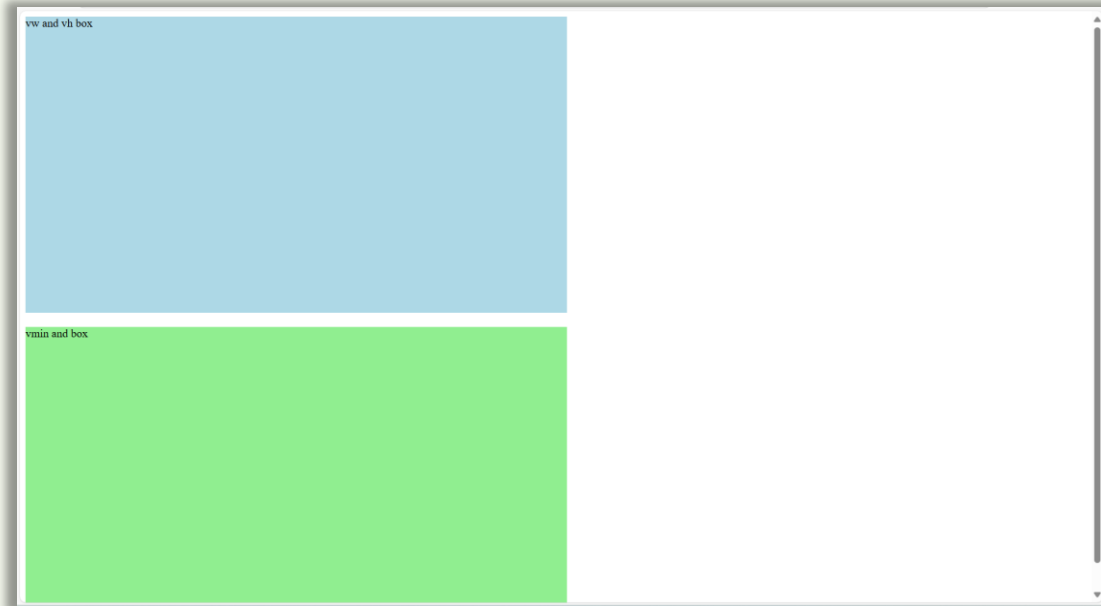
Viewport: 1200px width × 800px height

1vmin = 1% of 800px = 8px

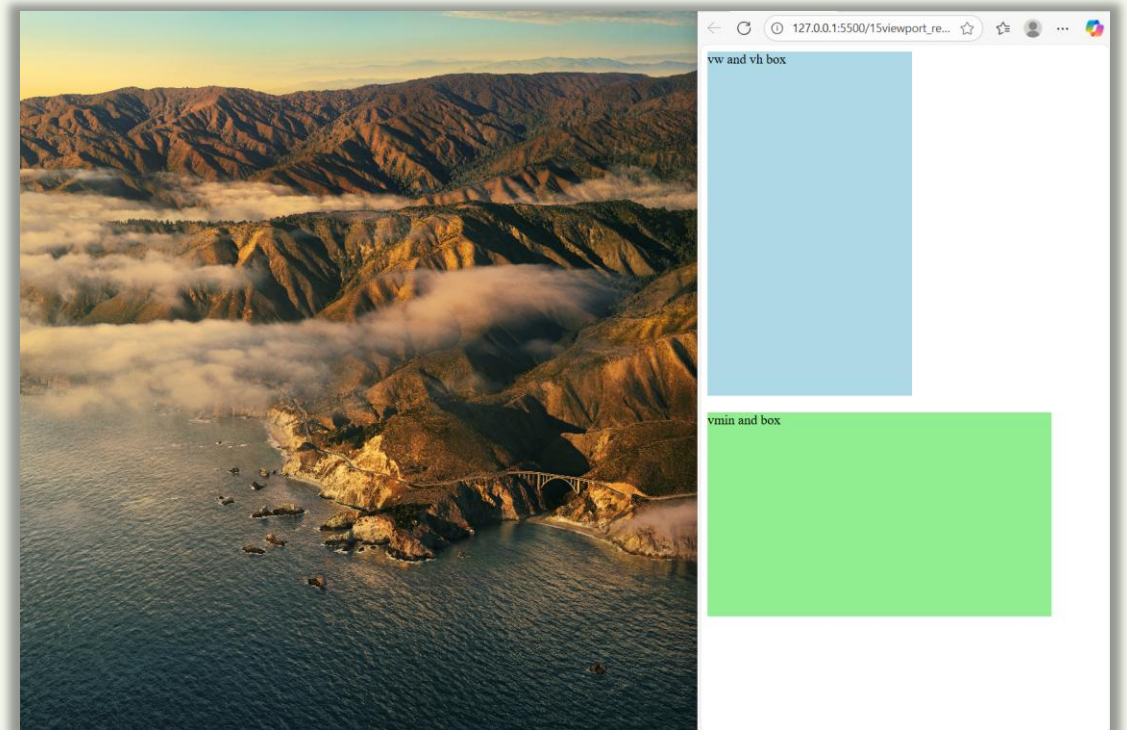
1vmax = 1% of 1200px = 12px

Dimensions and Length

Relative Length



Viewport-Relative Units



Dimensions and Length

Relative Length

Viewport-Relative Units

Container-relative units divide the available container space and are useful in container queries for responsive font sizing

Dimensions and Length

Relative Length

Viewport-Relative Units

Container-relative units divide the available container space and are useful in container queries for responsive font sizing

units	relative to
cqw	1% of the container's width.
cqh	1% of the container's height.
cqmin	1% of the container's smaller dimension.
cqmax	1% of the container's larger dimension.

Dimensions and Length

Relative Length

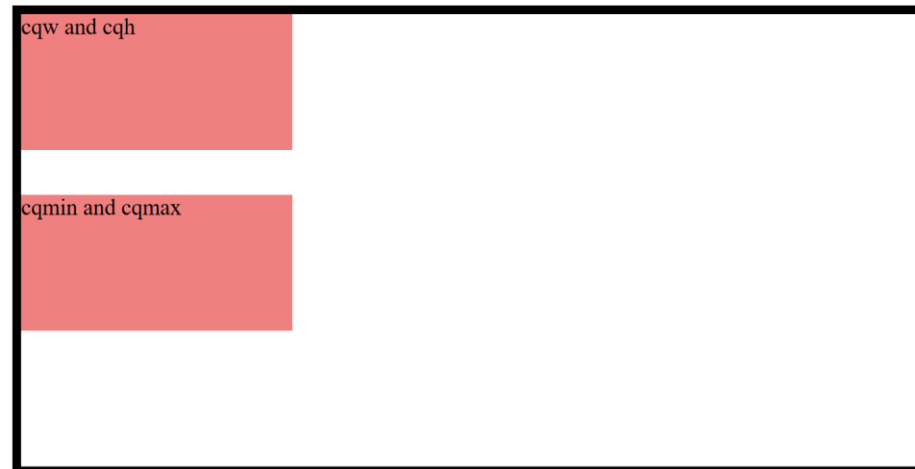
Viewport-Relative Units

```
<style>
  .container {
    container-type: size;
    width: 200px;
    height: 100px;
    border: 2px solid black;
    font-size: 5px;
  }

  .box1 {
    width: 30cqw;
    height: 30cqh;
    background-color: lightcoral;
    margin-bottom: 10px;
  }

  .box2 {
    width: 30cqmax;
    height: 30cqmin;
    background-color: lightcoral;
  }
</style>
```

```
<body>
  <div class="container">
    <div class="box1">cqw and cqh</div>
    <div class="box2">cqmin and cqmax</div>
  </div>
</body>
```



Dimensions and Length

Relative Length

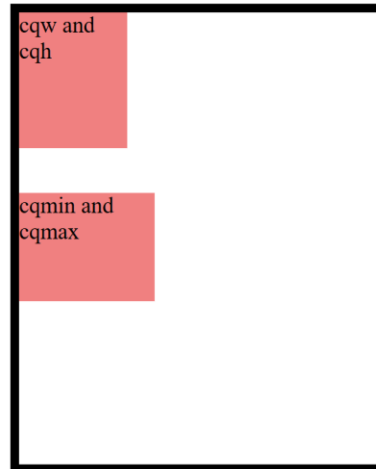
Viewport-Relative Units

```
<style>
  .container {
    container-type: size;
    width: 80px;
    height: 100px;
    border: 2px solid black;
    font-size: 5px;
  }

  .box1 {
    width: 30cqw;
    height: 30cqh;
    background-color: lightcoral;
    margin-bottom: 10px;
  }

  .box2 {
    width: 30cqmax;
    height: 30cqmin;
    background-color: lightcoral;
  }
</style>
```

```
<body>
  <div class="container">
    <div class="box1">cqw and cqh</div>
    <div class="box2">cqmin and cqmax</div>
  </div>
</body>
```



CSS

Flexbox

CSS

Flexbox



CSS

Flexbox

- Flexbox is a layout model for **one-dimensional** content

CSS

Flexbox

- Flexbox is a layout model for **one-dimensional** content
- It organizes content in a **single row** or **column**

CSS

Flexbox

- Flexbox is a layout model for **one-dimensional** content
- It organizes content in a **single row** or **column**
- It efficiently provides the best layout for items with **varying sizes**

Flexbox

Main Axis and Cross Axis

Flexbox

Main Axis and Cross Axis

- **Main axis** is the primary direction in which flex items are laid out (horizontal by default)

Flexbox

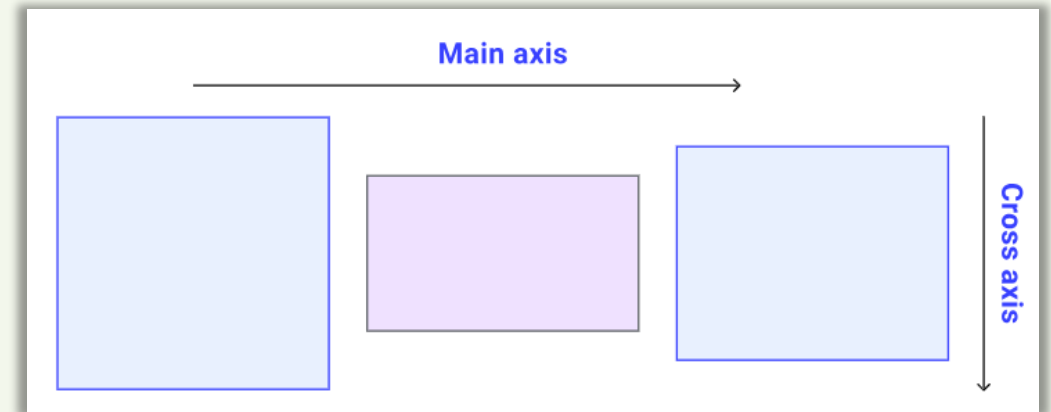
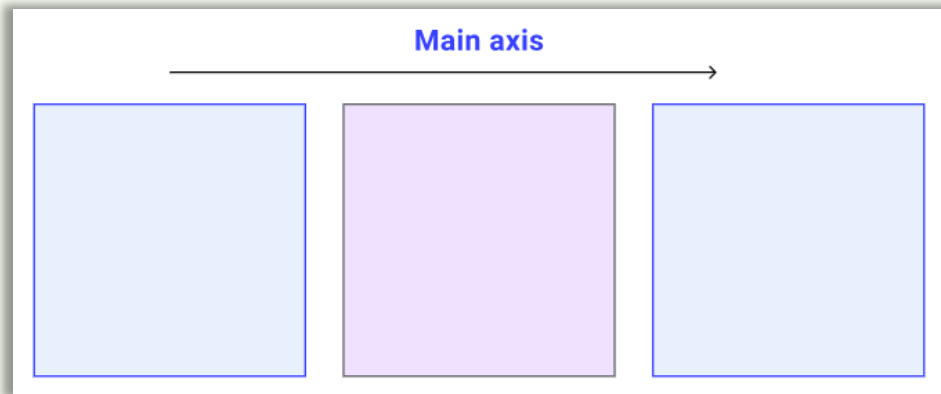
Main Axis and Cross Axis

- **Main axis** is the primary direction in which flex items are laid out (horizontal by default)
- **Cross axis** is perpendicular to the main axis (vertical by default)

Flexbox

Main Axis and Cross Axis

- **Main axis** is the primary direction in which flex items are laid out (horizontal by default).
- **Cross axis** is perpendicular to the main axis (vertical by default).



Flexbox

Main Axis and Cross Axis

Flexbox

Main Axis and Cross Axis

- The direction of the main axis is determined by the **flex-direction** property

Flexbox

Main Axis and Cross Axis

- The direction of the main axis is determined by the **flex-direction** property
- Flex items align along the main axis using **justify-content**

Flexbox

Main Axis and Cross Axis

- The direction of the main axis is determined by the **flex-direction** property
- Flex items align along the main axis using **justify-content**
- Flex items align along the cross axis using **align-items**

Flexbox

flex-direction property

Flexbox

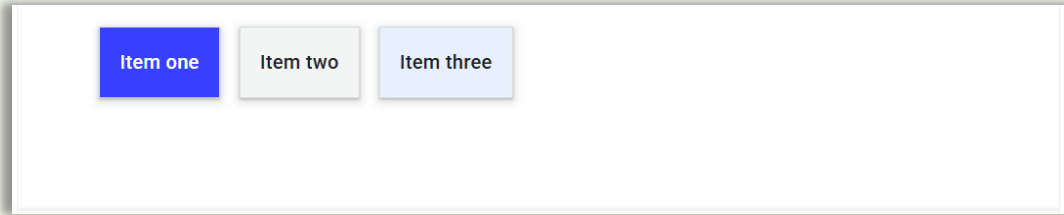
flex-direction property

Values

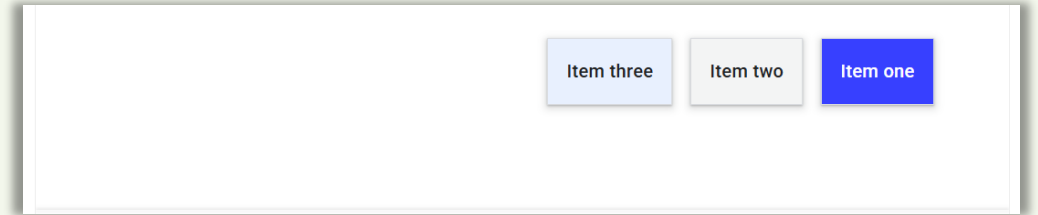
- **row**: the items lay out as a row
- **row-reverse**: the items lay out as a row from the end of the flex container
- **column**: the items lay out as a column
- **column-reverse** : the items lay out as a column from the end of the flex container

Flexbox

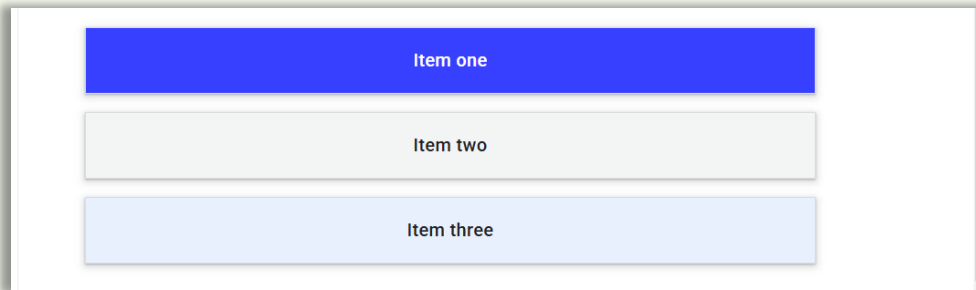
flex-direction property



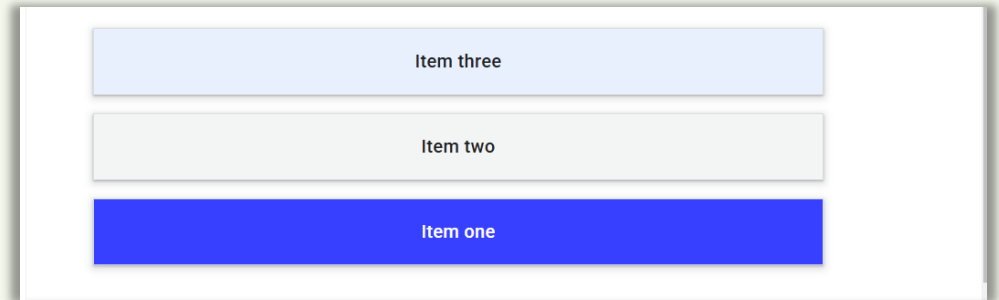
flex-direction: row;



flex-direction: row-reverse;



flex-direction: column;



flex-direction: column-reverse;

Flexbox

justify-content property

Values

- **flex-start**: Items are positioned at the beginning of the container
- **flex-end**: Items are positioned at the end of the container
- **center**: Items are positioned in the center of the container
- **space-between** : Items will have space between them
- **space-around**: Items will have space before, between, and after them
- **space-evenly**: Items will have equal space around them

Flexbox

justify-content property

☒ `justify-content: flex-start;`

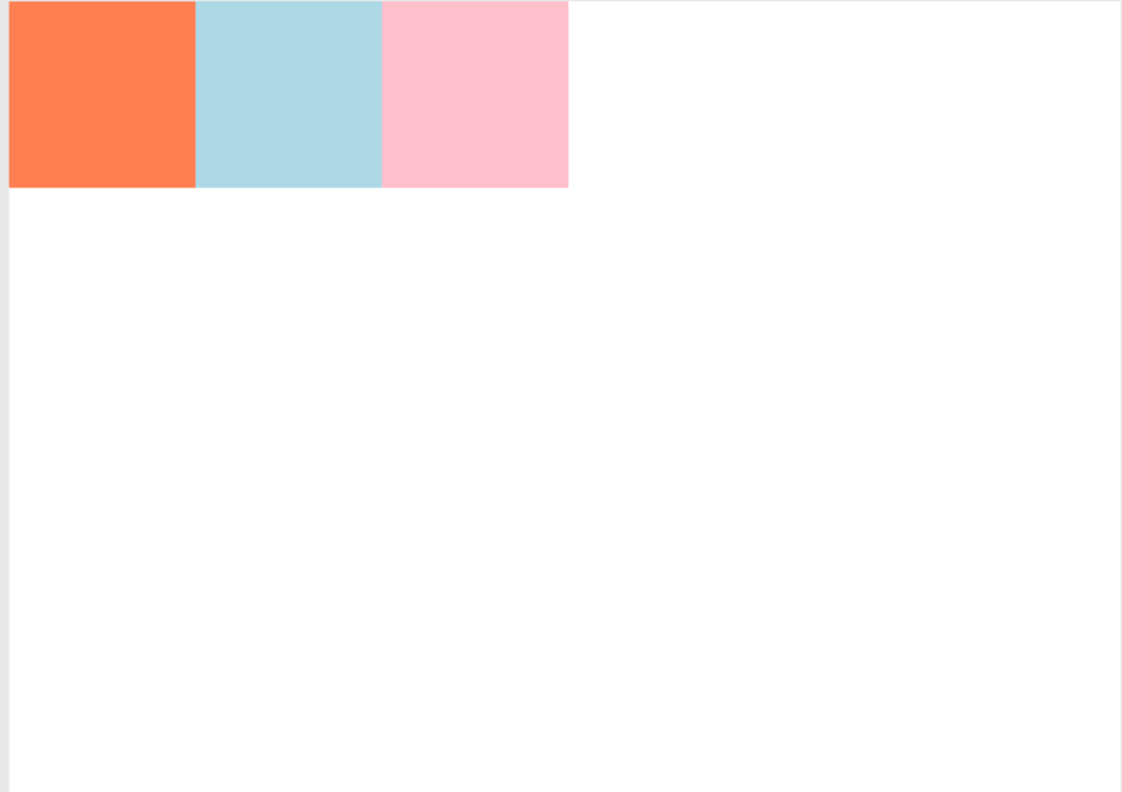
☐ `justify-content: flex-end;`

☐ `justify-content: center;`

☐ `justify-content: space-between;`

☐ `justify-content: space-around;`

☐ `justify-content: space-evenly;`



Flexbox

justify-content property

☐ `justify-content: flex-start;`

☒ `justify-content: flex-end;`

☐ `justify-content: center;`

☐ `justify-content: space-between;`

☐ `justify-content: space-around;`

☐ `justify-content: space-evenly;`



Flexbox

justify-content property

☐ `justify-content: flex-start;`

☐ `justify-content: flex-end;`

☒ `justify-content: center;`

☐ `justify-content: space-between;`

☐ `justify-content: space-around;`

☐ `justify-content: space-evenly;`



Flexbox

justify-content property

- ☐ `justify-content: flex-start;`
- ☐ `justify-content: flex-end;`
- ☐ `justify-content: center;`
- ☒ `justify-content: space-between;`
- ☐ `justify-content: space-around;`
- ☐ `justify-content: space-evenly;`



Flexbox

justify-content property

- ☐ `justify-content: flex-start;`
- ☐ `justify-content: flex-end;`
- ☐ `justify-content: center;`
- ☐ `justify-content: space-between;`
- ☒ `justify-content: space-around;`
- ☐ `justify-content: space-evenly;`



Flexbox

justify-content property

- ☐ `justify-content: flex-start;`

- ☐ `justify-content: flex-end;`

- ☐ `justify-content: center;`

- ☐ `justify-content: space-between;`

- ☐ `justify-content: space-around;`

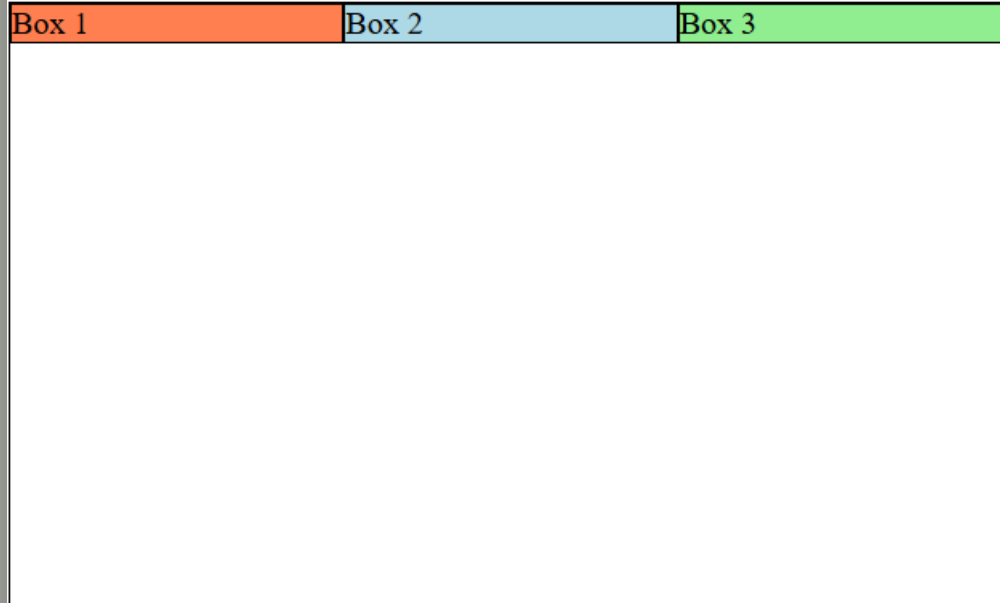
- ☒ `justify-content: space-evenly;`



Flexbox

align-items property

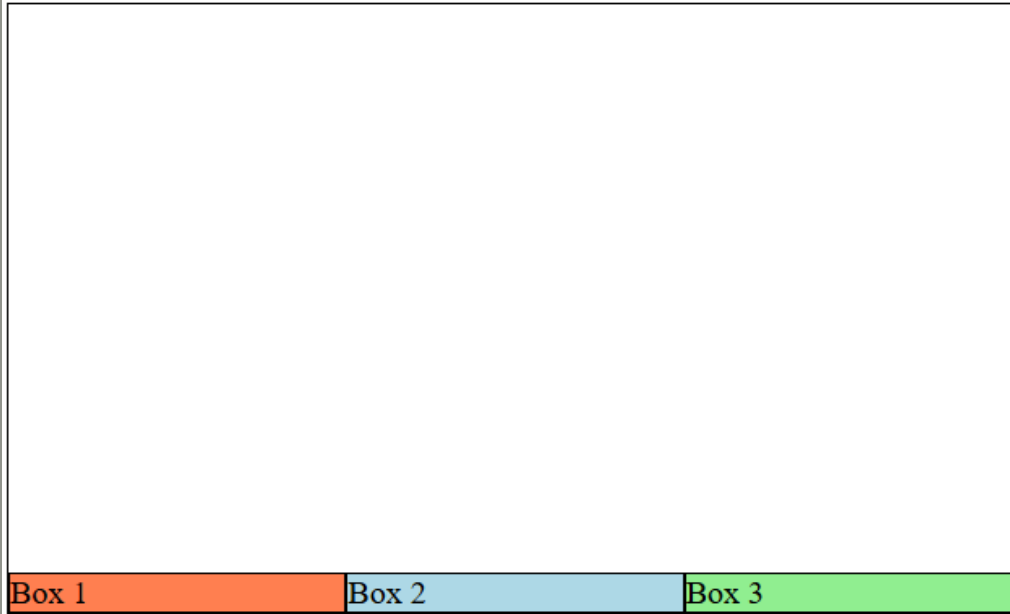
flex-start



Flexbox

align-items property

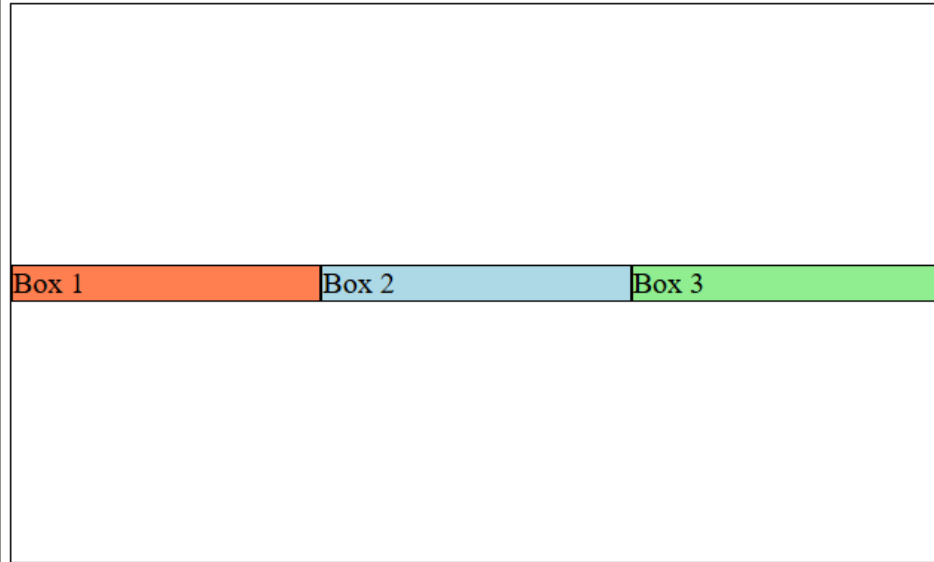
flex-end



Flexbox

align-items property

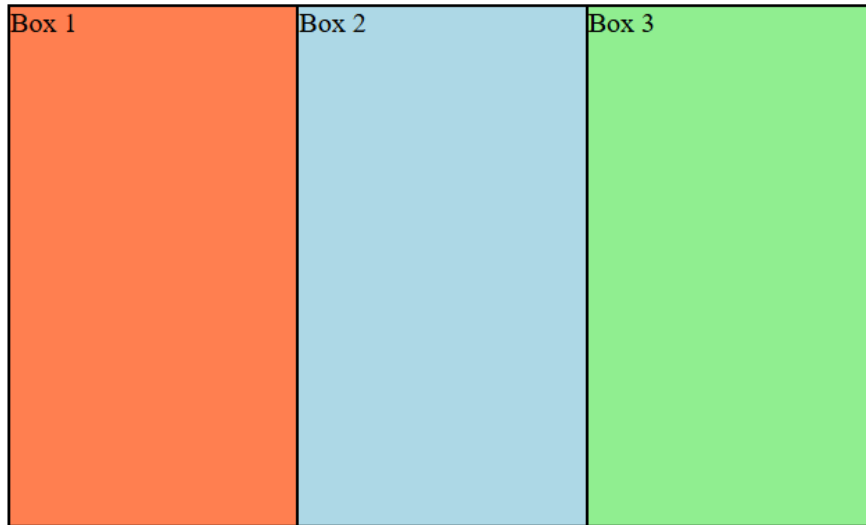
center



Flexbox

align-items property

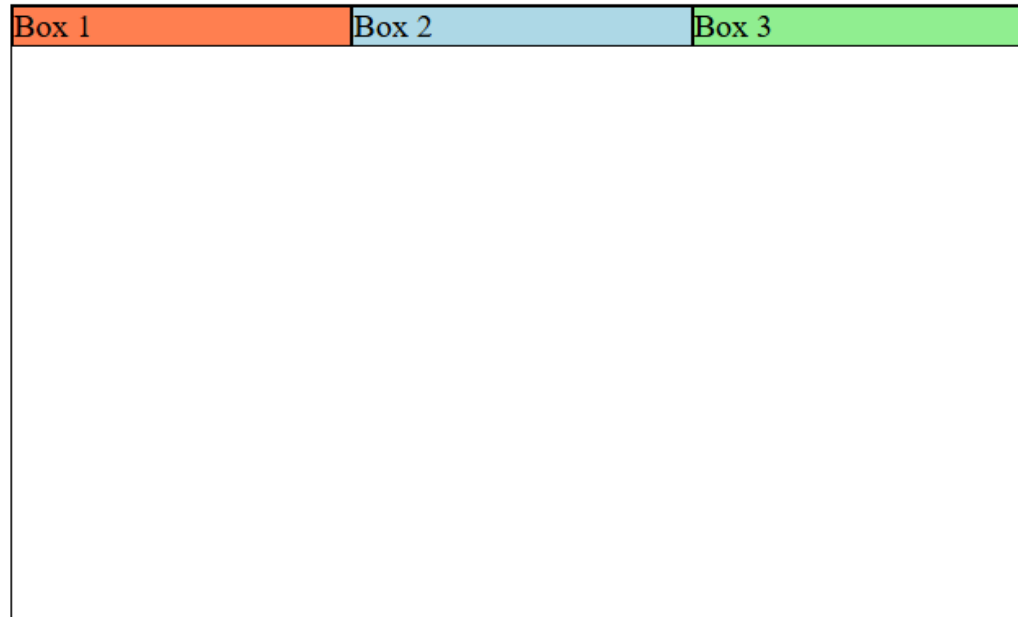
stretch



Flexbox

align-items property

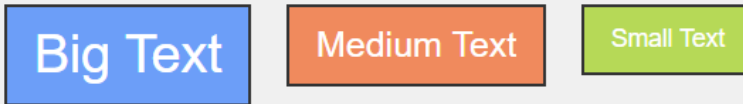
baseline



Flexbox

align-items property

flex-start



Flexbox

align-items property

baseline

Big Text

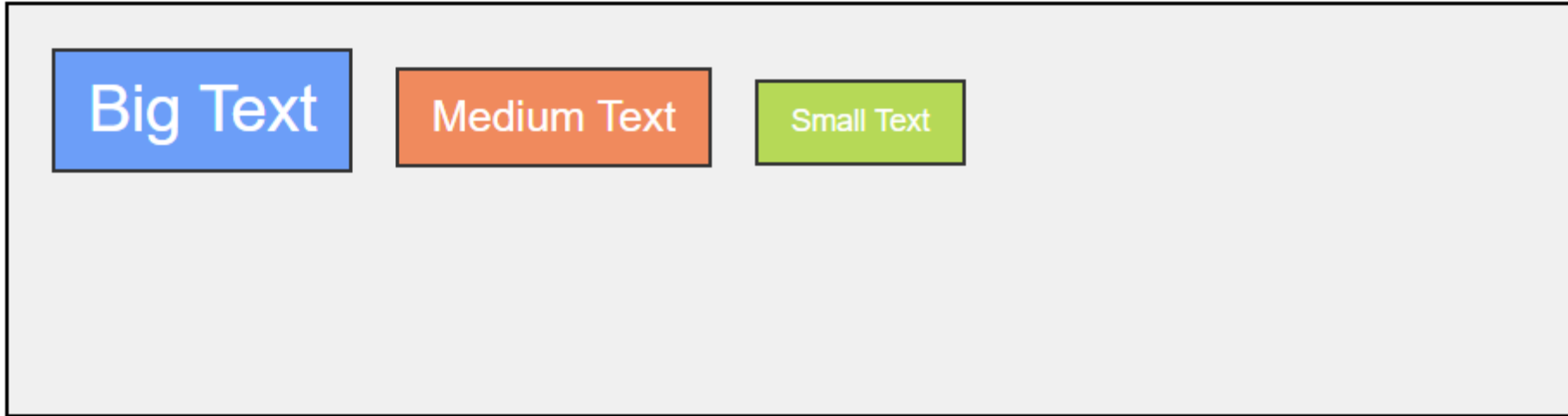
Medium Text

Small Text

Flexbox

align-items property

baseline



Notice how the text in all boxes lines up along the same baseline.

Flexbox

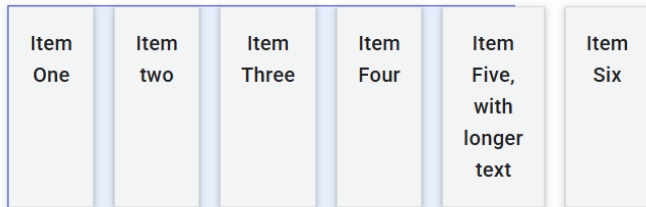
Wrapping Flex Items

- The initial value of the flex-wrap property is **nowrap**
- Items shrink to min-content by default; use **flex-wrap: wrap** to enable wrapping

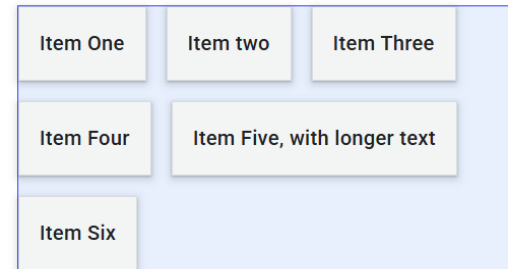
Flexbox

Wrapping Flex Items

- The initial value of the flex-wrap property is **nowrap**
- Items shrink to min-content by default; use **flex-wrap: wrap** to enable wrapping



flex-wrap: nowrap;



flex-wrap: wrap;

Flexbox

Game for Learning CSS Flexbox

← → ↻ 📄 flexboxfroggy.com ☆ 🗑️ 👤 ⋮

FLEXBOX FROGGY

Level 1 of 24 ▾

Welcome to Flexbox Froggy, a game where you help Froggy and friends by writing CSS code! Guide this frog to the lilypad on the right by using the `justify-content` property, which aligns items horizontally and accepts the following values:

- `flex-start`: Items align to the left side of the container.
- `flex-end`: Items align to the right side of the container.
- `center`: Items align at the center of the container.
- `space-between`: Items display with equal spacing between them.
- `space-around`: Items display with equal spacing around them.

For example, `justify-content: flex-end;` will move the frog to the right.

1 #pond {
2 display: flex;
3
4 }
5
6
7
8
9
10

Next

Flexbox Froggy is created by [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Settings](#)

Want to learn more CSS? Play [Grid Garden](#) or [Anchoreum](#).

←

Ads by Google

Send feedback

Why this ad? ⓘ





CSS
Grid

CSS Grid

a layout system for building **two-dimensional** layouts on the web

CSS Grid

a layout system for building two-dimensional layouts on the web

use **display: grid** to turn an element into a grid container

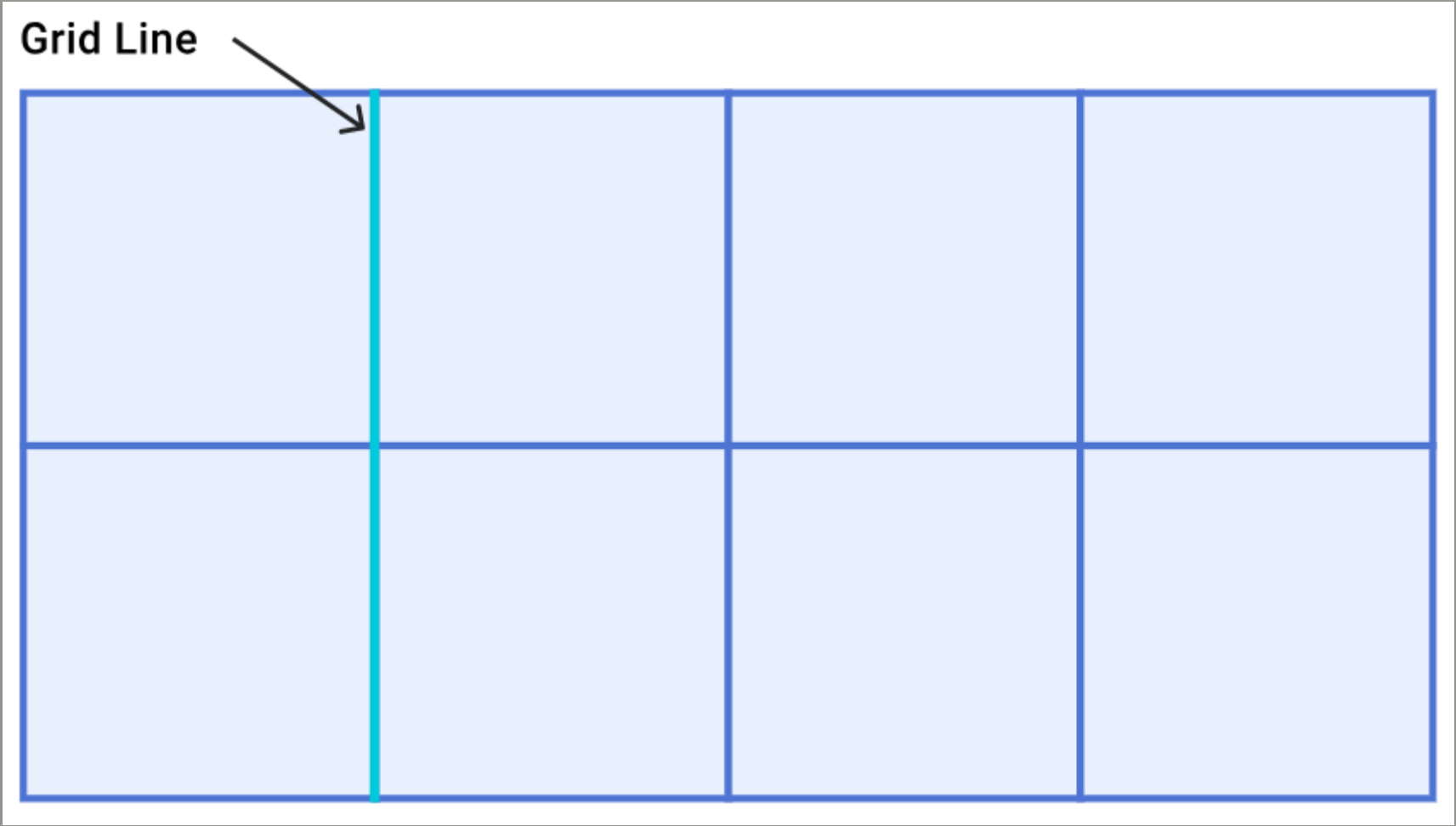
Grid

Grid Lines

- A grid is composed of horizontal and vertical lines.
- If there are 4 columns, there will be 5 column lines (one more than the number of columns).
- Lines are numbered starting from 1.

Grid

Grid Lines



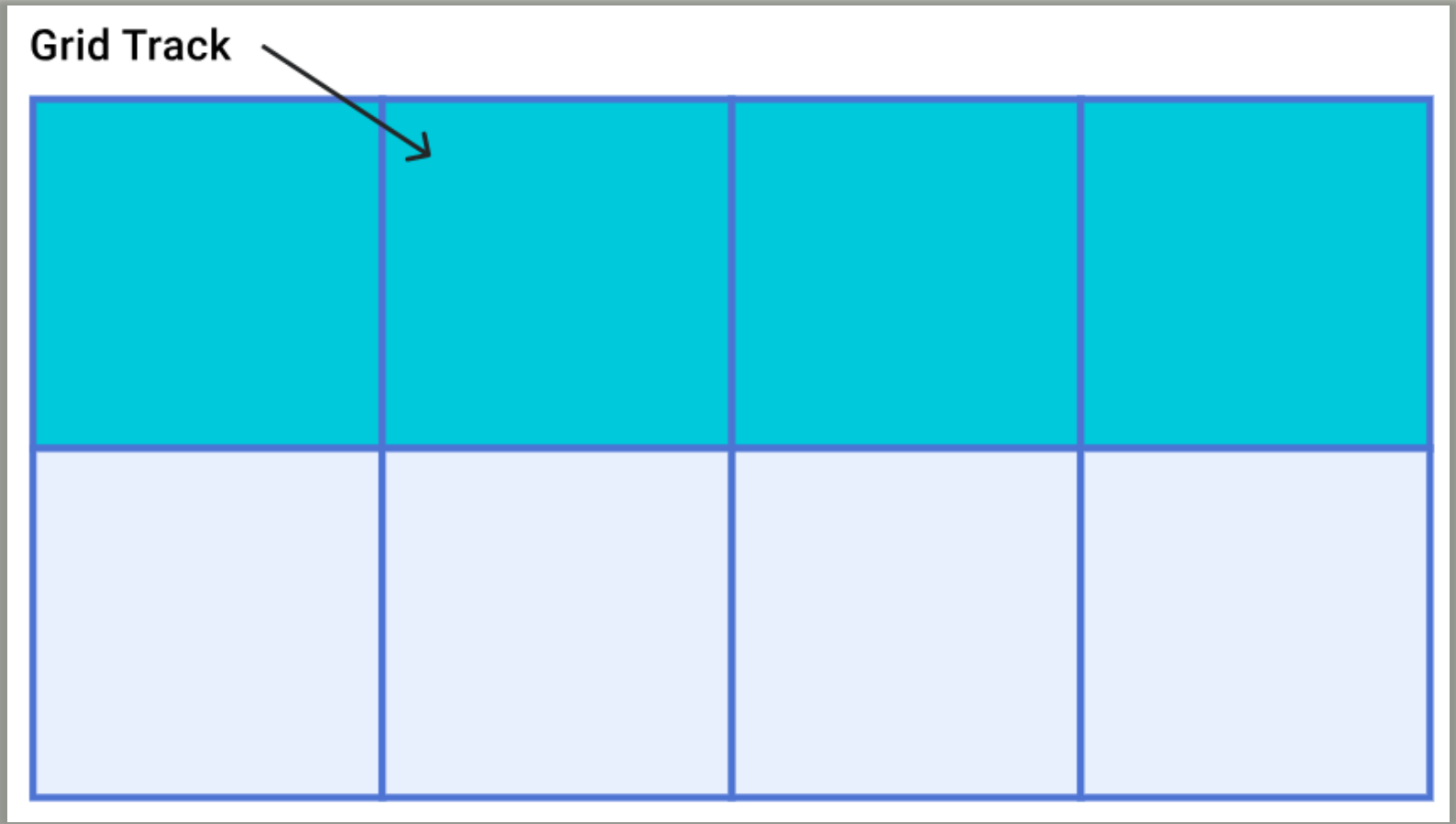
Grid

Grid Tracks

- A **track** is the space between two grid lines.
- A **row track** lies between two row lines.
- A **column track** lies between two column lines.
- Tracks are created by **assigning sizes** to them when defining the grid.

Grid

Grid Tracks



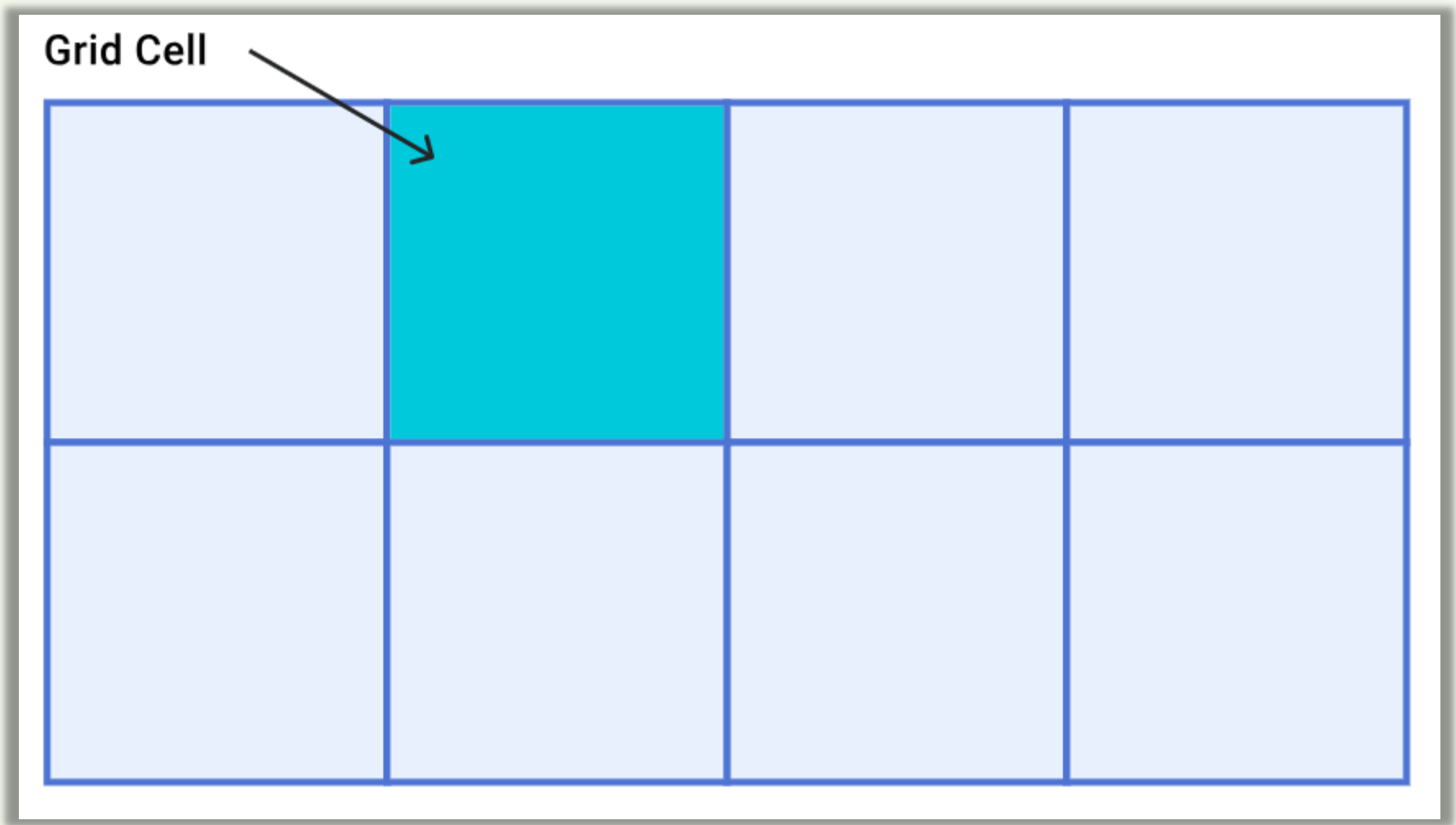
Grid

Grid Cell

- A grid cell is the **smallest unit** in a grid, formed by the **intersection** of row and column tracks.
- It is similar to a table cell or spreadsheet cell.
- By default, items are **automatically** placed into individual grid cells if not manually positioned.

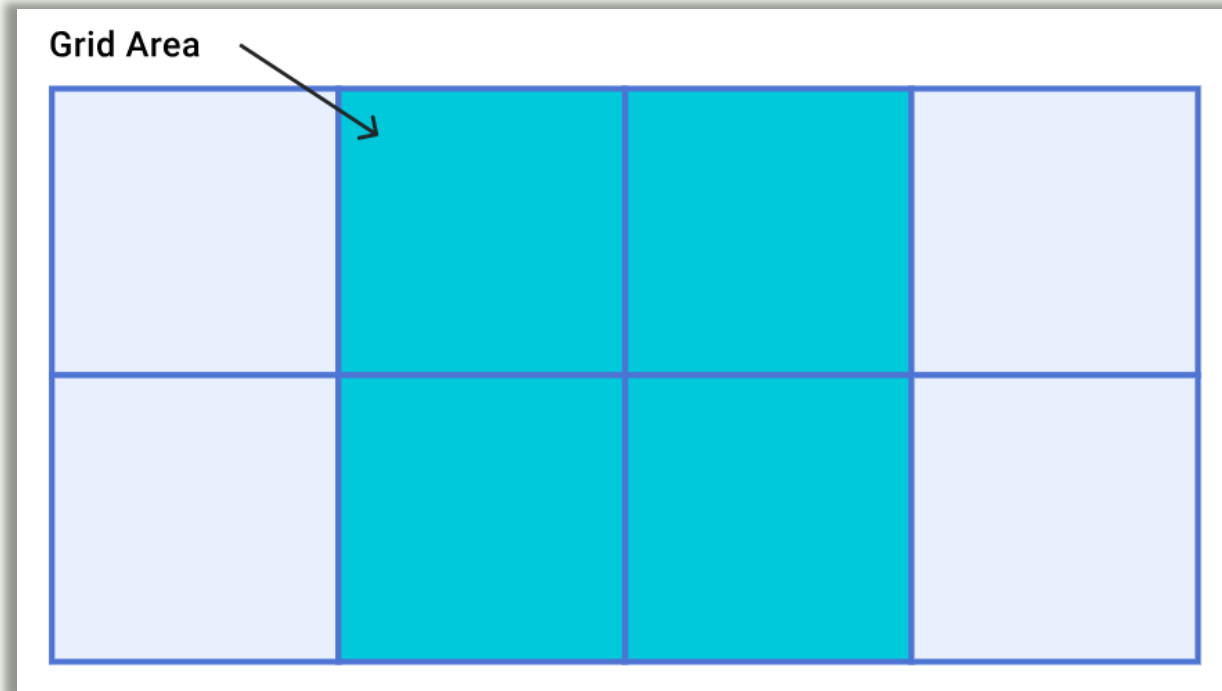
Grid

Grid Cell



Grid Grid Area

- Several grid cells together. Grid areas are created by causing an item to span over multiple tracks.



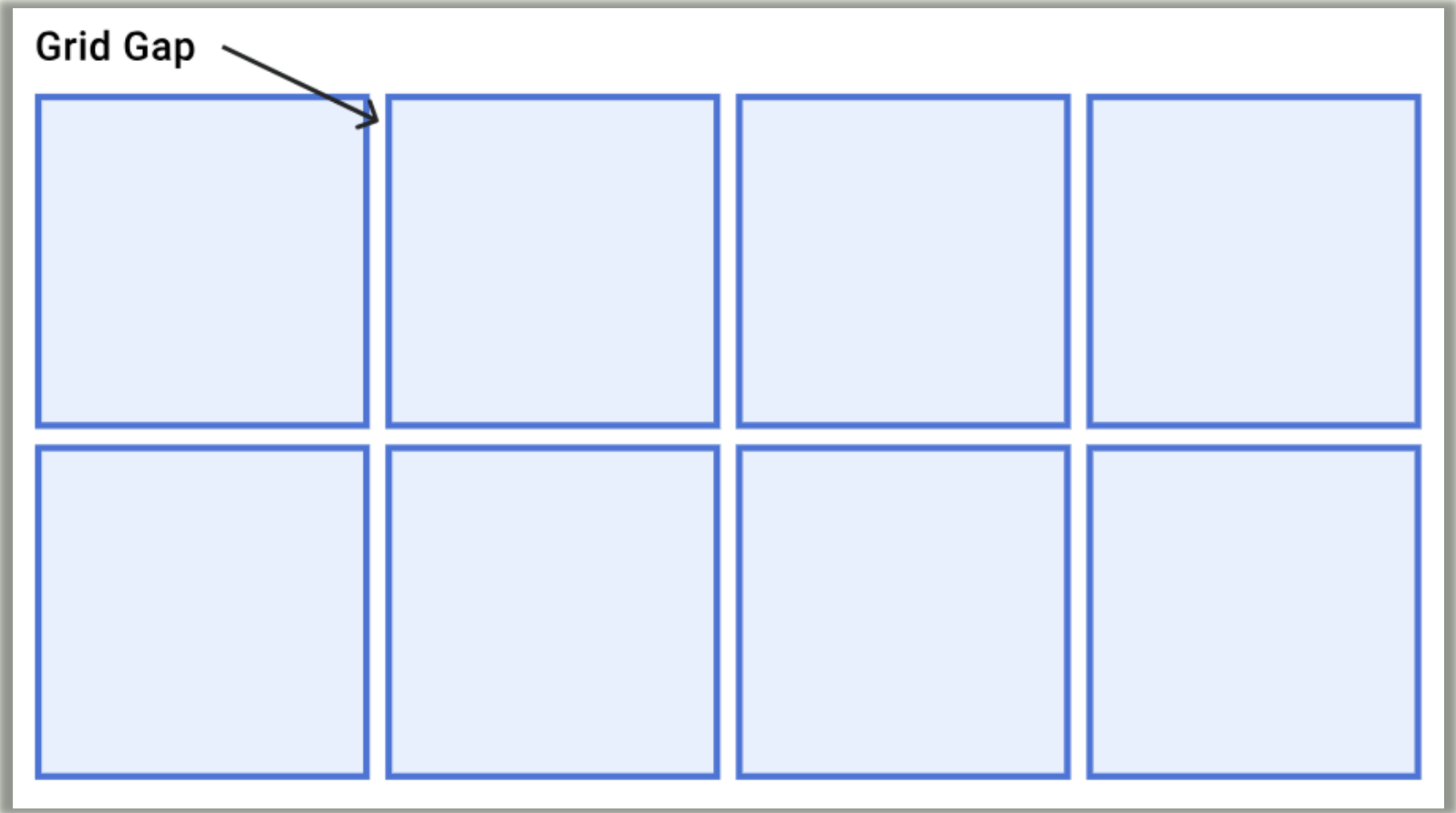
Grid

Grid Gap

- A gutter (or alley) is the space between tracks and behaves like a regular track for sizing.
- Content can't be placed directly into a gap, but grid items can span across it.

Grid

Grid Gap





Lorem Ipsum Dolor Sit Amet

Cras Justo Odio, Dapibus Ac Facilisis In,
Egestas Eget Quam. Donec Id Elit Non Mi Porta
Gravida At Eget Metus. Donec Sed Odio Dui.
Curabitur Blandit Tempus Porttitor. Etiam Porta
Sem Malesuada Magna Mollis Euismod.
Curabitur Blandit Tempus Porttitor. Donec
Ullamcorper Nulla Non Metus Auctor Fringilla.

Cras Justo Odio, Dapibus Ac Facilisis In,
Egestas Eget Quam. Donec Id Elit Non Mi Porta
Gravida At Eget Metus. Donec Sed Odio Dui.
Curabitur Blandit Tempus Porttitor. Etiam Porta
Sem Malesuada Magna Mollis Euismod.
Curabitur Blandit Tempus Porttitor. Donec
Ullamcorper Nulla Non Metus Auctor Fringilla.

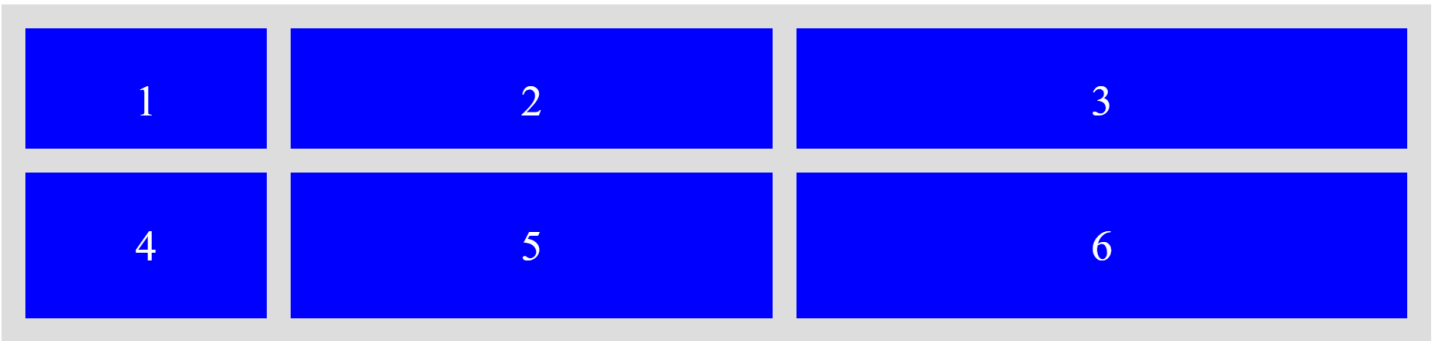
Grid

```
<body>
  <h2>CSS Grid Track Sizes</h2>
  <div class="grid">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
    <div class="item">4</div>
    <div class="item">5</div>
    <div class="item">6</div>
  </div>
</body>
```

```
<style>
  .grid {
    display: grid;
    grid-template-columns: 100px 200px 1fr;
    /* 3 columns with different sizes */
    grid-template-rows: 50px 1fr;
    /* 2 rows */
    gap: 10px;
    background-color: #ddd;
    padding: 10px;
  }

  .item {
    background-color: blue;
    color: white;
    padding: 20px;
    text-align: center;
    font-size: 18px;
  }
</style>
```

CSS Grid Track Sizes



CSS

Positioning

CSS

Positioning

- a layout mechanism that controls how an element behaves in the normal document flow and how it relates to other elements
- the position property determines this behavior and supports several values

CSS

Positioning

- a layout mechanism that controls how an element behaves in the normal document flow and how it relates to other elements
- the position property determines this behavior and supports several values

Static

Relative

Absolute

Fixed

Sticky

CSS

Positioning

Static

The element follows the normal document flow and ignores positioning offsets.

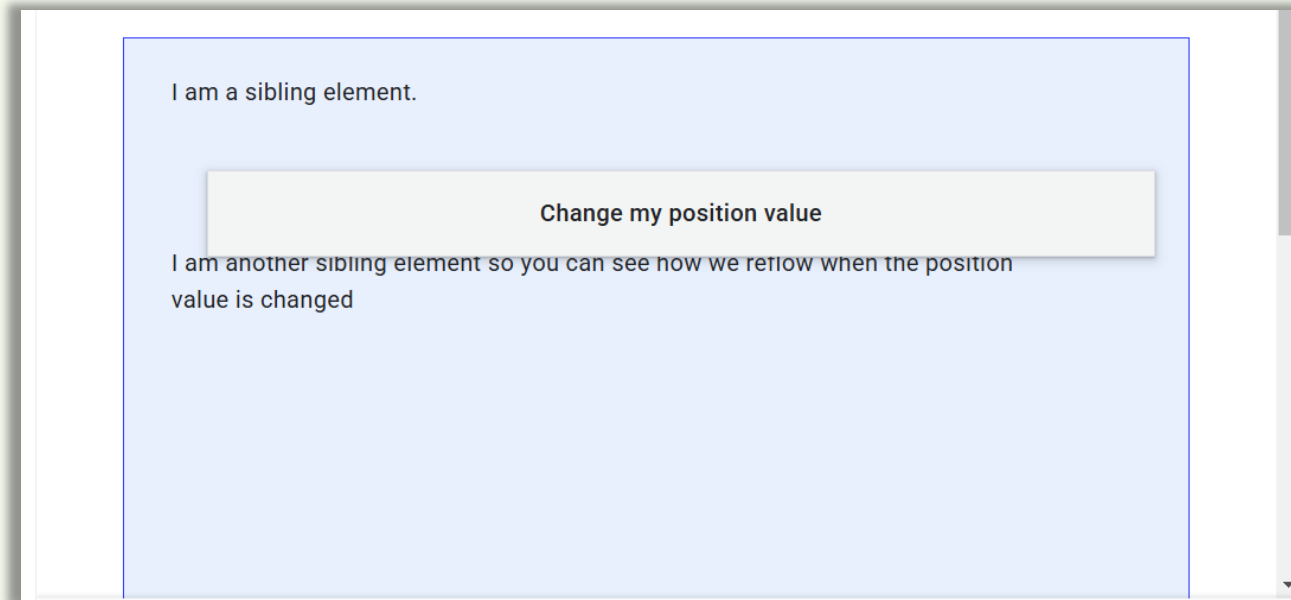


CSS

Positioning

Relative

The element stays in flow but can be offset from its original position.

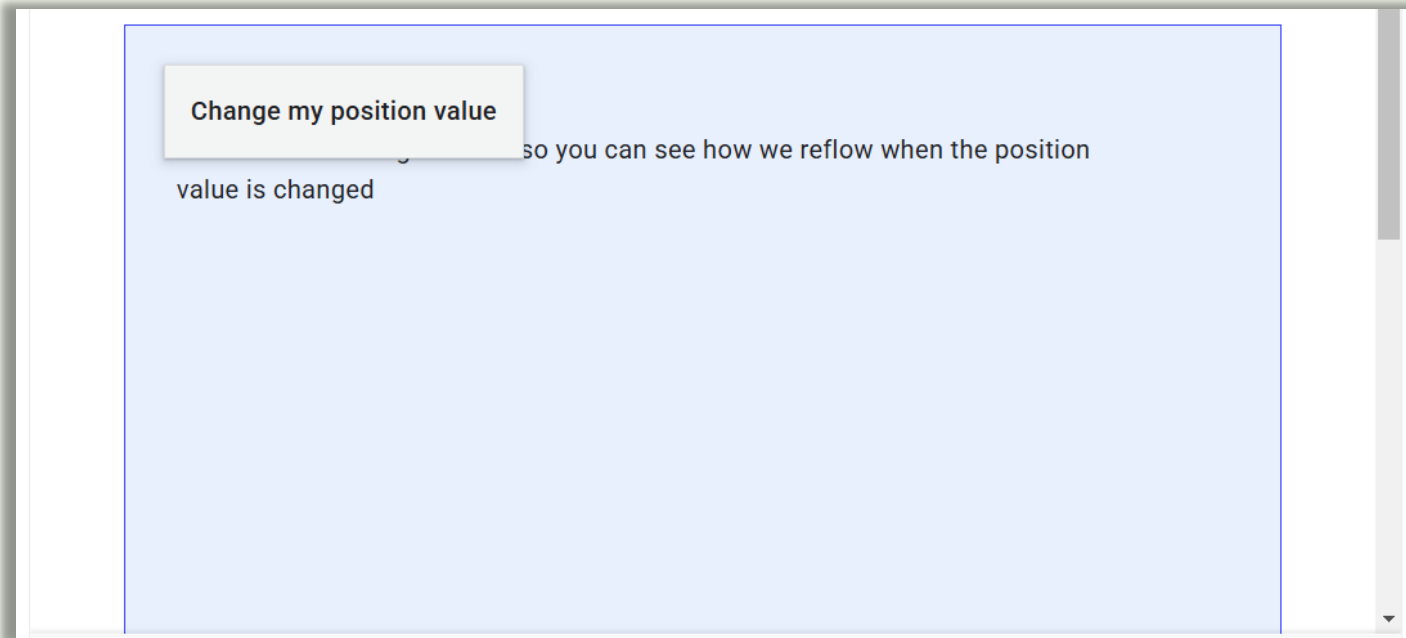


CSS

Positioning

Absolute

The element is removed from flow and positioned relative to the nearest non-static ancestor.

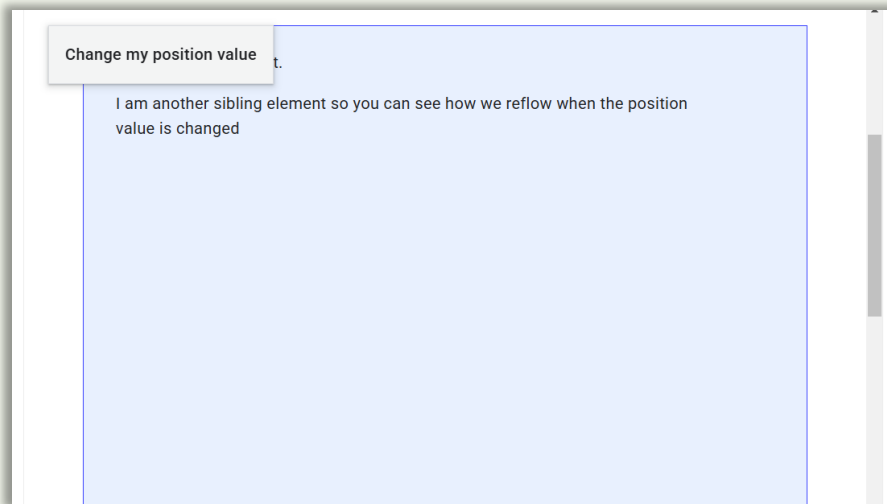


CSS

Positioning

Fixed

The element is removed from flow and positioned relative to the viewport, staying fixed during scroll.

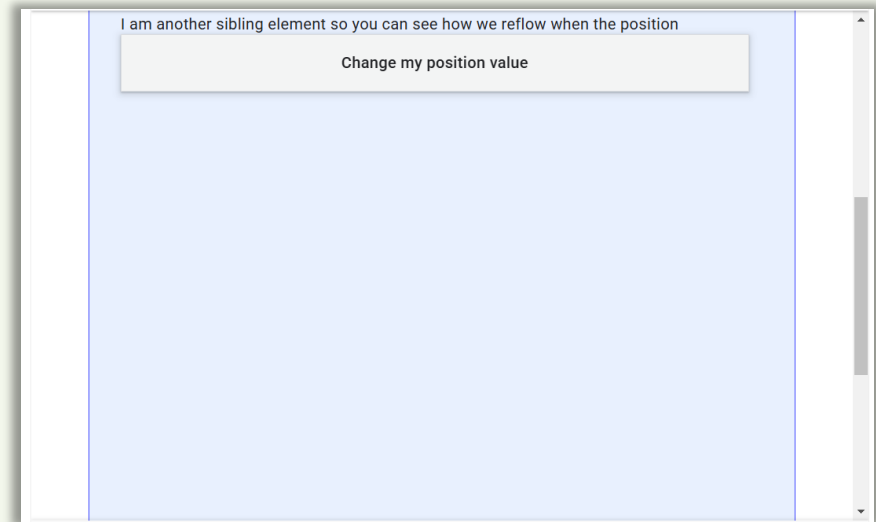
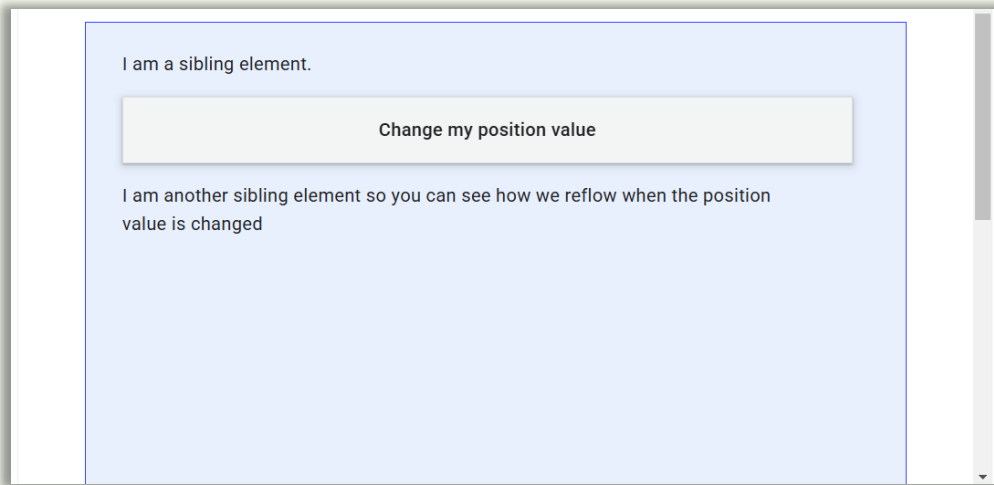


CSS

Positioning

Sticky

The element acts like relative until a scroll threshold is reached, then behaves like fixed.



CSS

Z-index

CSS

Z-index

- The z-index property controls the **stacking order** of elements along the z-axis, which represents depth in the browser (front to back).

CSS

Z-index

- The z-index property controls the **stacking order** of elements along the z-axis, which represents depth in the browser (front to back).

In web layout:

X-axis = horizontal (left to right)

Y-axis = vertical (top to bottom)

Z-axis = stacking (closer or farther from the viewer)

CSS

Z-index

- The z-index property controls the **stacking order** of elements along the z-axis, which represents depth in the browser (front to back).

In web layout:

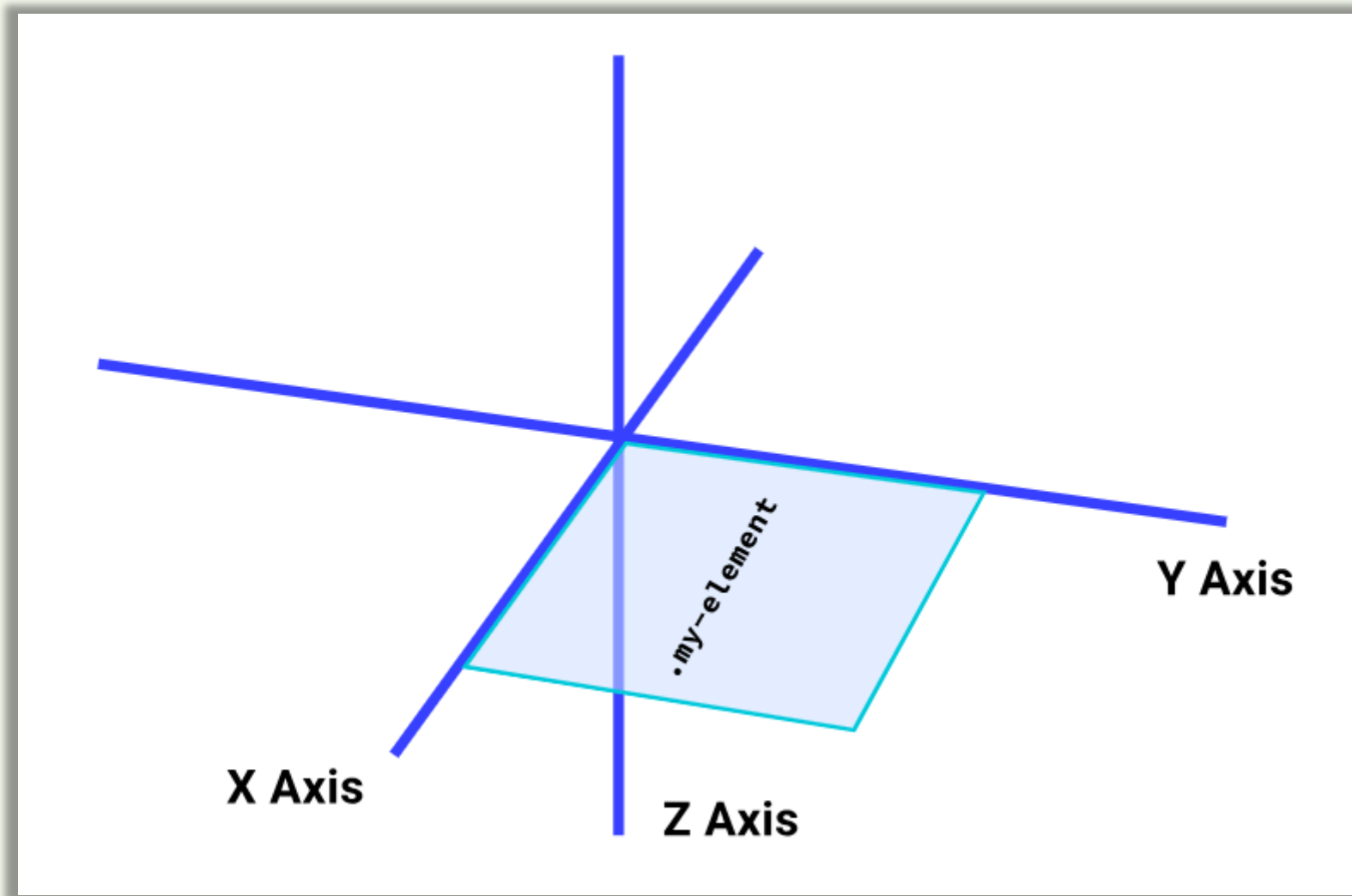
X-axis = horizontal (left to right)

Y-axis = vertical (top to bottom)

Z-axis = stacking (closer or farther from the viewer)

- A higher z-index places an element above those with lower values.

CSS Z-index



CSS

Z-index

The z-index property accepts positive or negative numbers to control stack order on the Z-axis.

CSS

Z-index

The z-index property accepts positive or negative numbers to control stack order on the Z-axis.

- Elements with a higher z-index appear **above** those with a lower one.
- If no z-index is set, the HTML source order determines stacking, later elements appear on **top** of earlier ones.

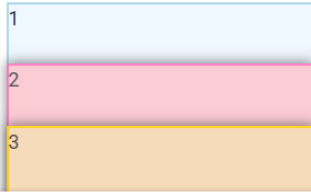
CSS

Z-index

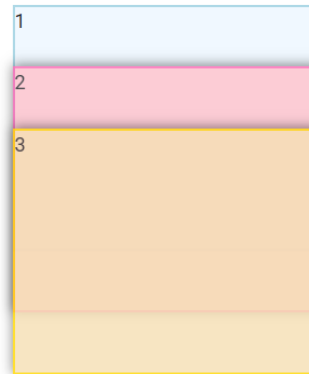
Default `z-index` behaviour

❗ If no value for `z-index` is set, the browser will use the document source order to dictate `z-index` instead.

This demo has 3 empty `<div>` elements, with negative margin, making them overlap. The later elements sit on top of the earlier elements.



This demo has 3 empty `<div>` elements, with negative margin, making them overlap. The later elements sit on top of the earlier elements.



Z-index

Negative z-index

- To set an element behind another element, add a negative value for z-index.



```
.my-element .child {  
  position: relative;  
  z-index: -1;  
}
```



```
.my-element {  
  position: relative;  
  z-index: 0;  
  background: rgb(232 240 254 / 0.4);  
}
```

Z-index

Stacking Contexts

- When a parent has a z-index and a position (not static), it creates a stacking context.

Z-index

Stacking Contexts

- When a parent has a z-index and a position (not static), it creates a stacking context.
 - Children stack relative to their parent's context.
 - High z-index on a child can't escape its context.
 - Parent with higher z-index stacks above others.

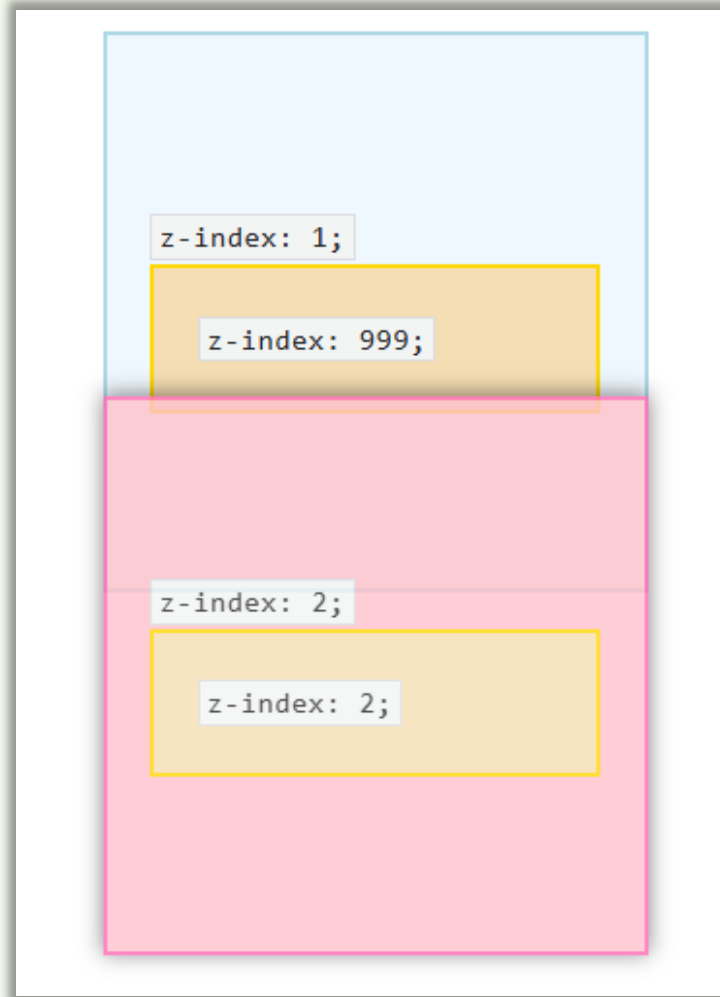
Z-index

Stacking Contexts

- When a parent has a z-index and a position (not static), it creates a stacking context.
 - Children stack relative to their parent's context.
 - High z-index on a child can't escape its context.
 - Parent with higher z-index stacks above others.
- z-index of children only affects stacking within their parent's context, not across contexts.

Z-index

Stacking Contexts



CSS

Media Queries

CSS

Media Queries

Media queries let you apply different CSS rules depending on device features like:

- Screen size (width/height)
- Orientation (portrait/landscape)
- Media type (screen/print)
- Resolution

CSS

Media Queries

Syntax

```
@media not|only mediatype and (mediafeature and|or|not mediafeature) {  
    CSS-Code;  
}
```

CSS

Media Queries

Syntax

```
@media not|only mediatype and (mediafeature and|or|not mediafeature) {  
  CSS-Code;  
}
```

- **not:** inverts the meaning of an entire media query.

CSS

Media Queries

Syntax

```
@media not|only mediatype and (mediafeature and|or|not mediafeature) {  
  CSS-Code;  
}
```

- **not:** inverts the meaning of an entire media query.
- **only:** prevents older browsers that do not support media queries with media features from applying the specified styles. It has no effect on modern browsers.

CSS

Media Queries

Syntax

```
@media not|only mediatype and (mediafeature and|or|not mediafeature) {  
  CSS-Code;  
}
```

- **not:** inverts the meaning of an entire media query.
- **only:** prevents older browsers that do not support media queries with media features from applying the specified styles. It has no effect on modern browsers.
- **and:** combines a media feature with a media type or other media features.
- ❖ They are all optional. However, if you use not or only, you must also specify a media type.

Media Queries

Media Types

describes the general category of a device

Media Queries

Media Types

describes the general category of a device

Value	Description
all	Default. Used for all media type devices
print	Used for printers
screen	Used for computer screens, tablets, smart-phones etc.

Media Queries

Media Features

Media features (in parentheses) are optional conditions for styling based on device capabilities like size, orientation, or resolution.

Media Queries

Media Features

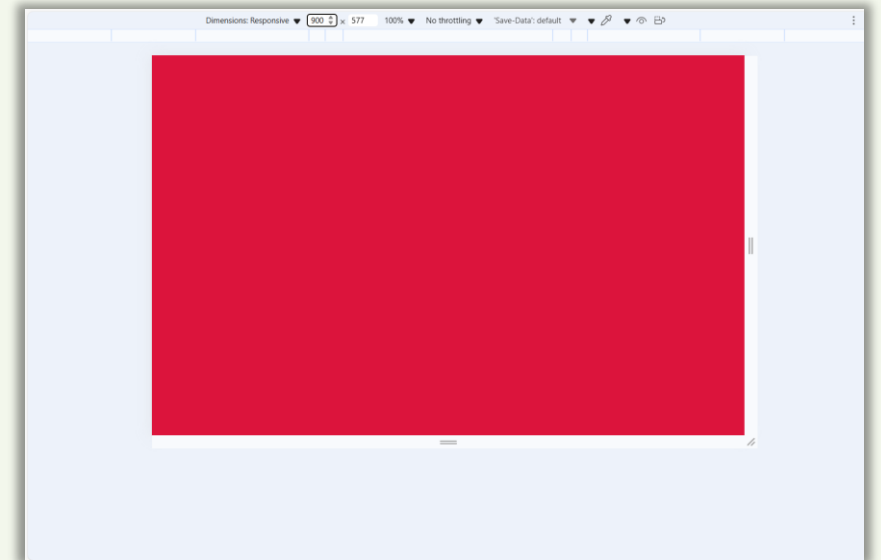
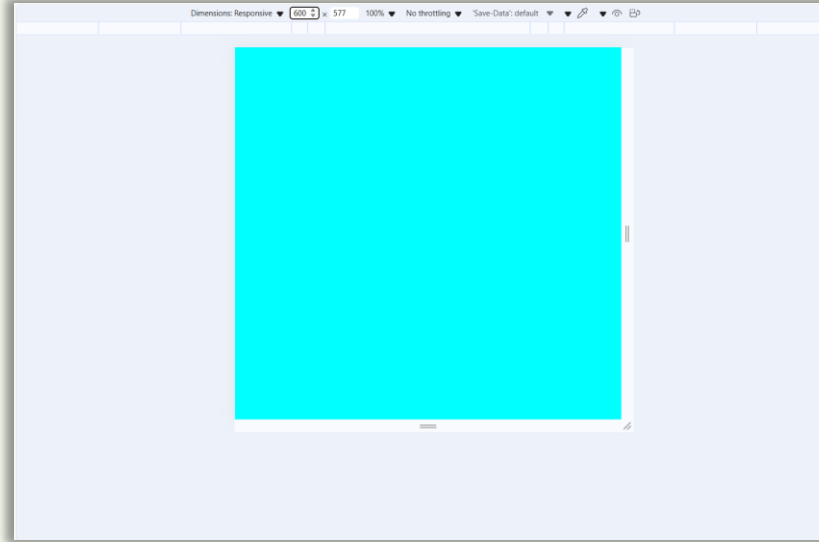
Value	Description
width	Viewport width
height	Viewport height
orientation	Landscape or portrait mode
aspect-ratio	Width-to-height ratio

Media Queries

Example 1

```
<style>
  body {
    background-color: ■ crimson;
  }

  @media only screen and (max-width:600px) {
    body {
      background-color: ■ aqua;
    }
  }
</style>
```



Media Queries

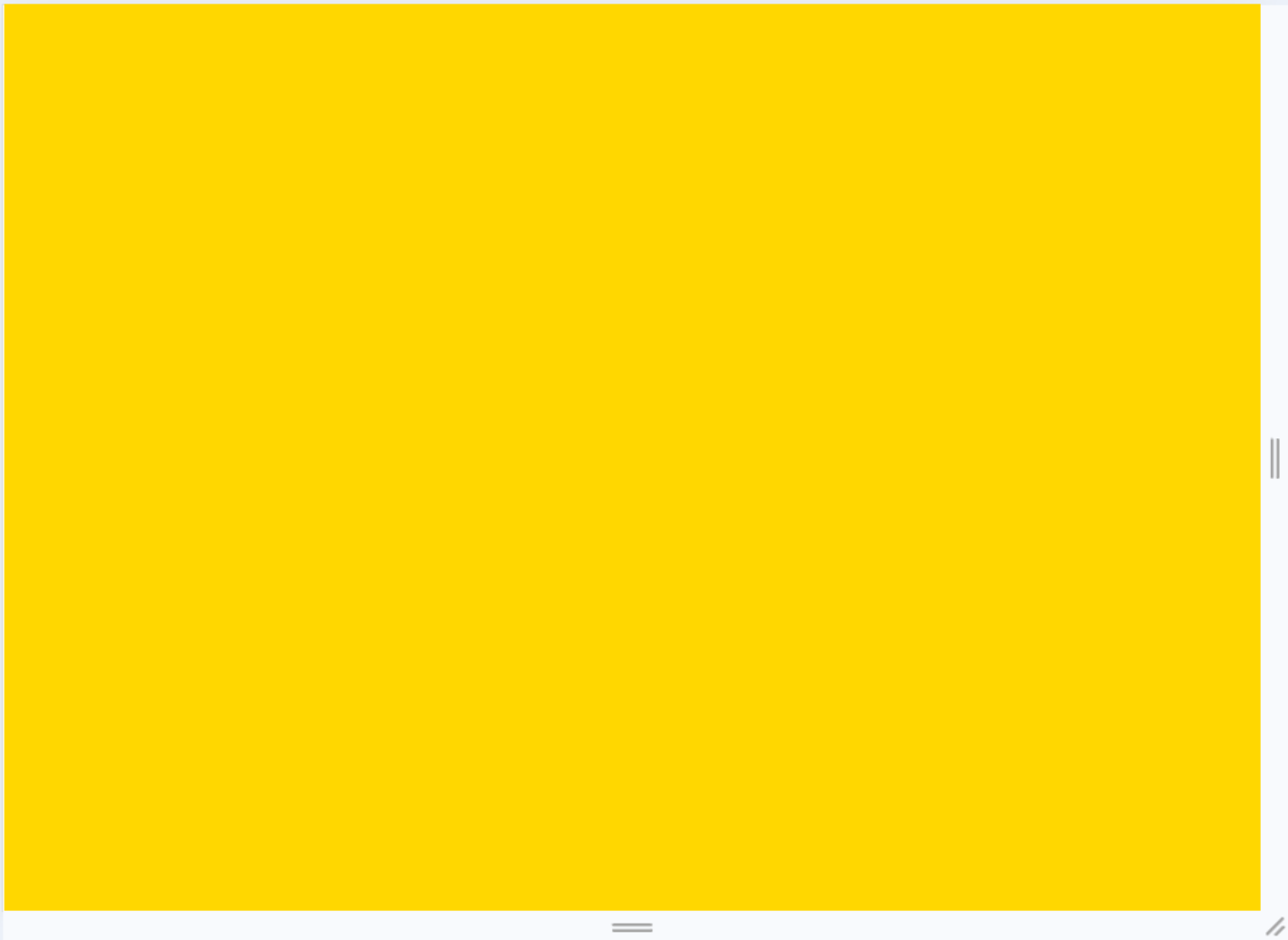
Example 2

```
<style>
  body {
    background-color: blueviolet;
  }

  @media screen and (min-width:400px) {
    body {
      background-color: turquoise;
    }
  }

  @media screen and (min-width:800px) {
    body {
      background-color: gold;
    }
  }
</style>
```

M



Media Queries

Example 3

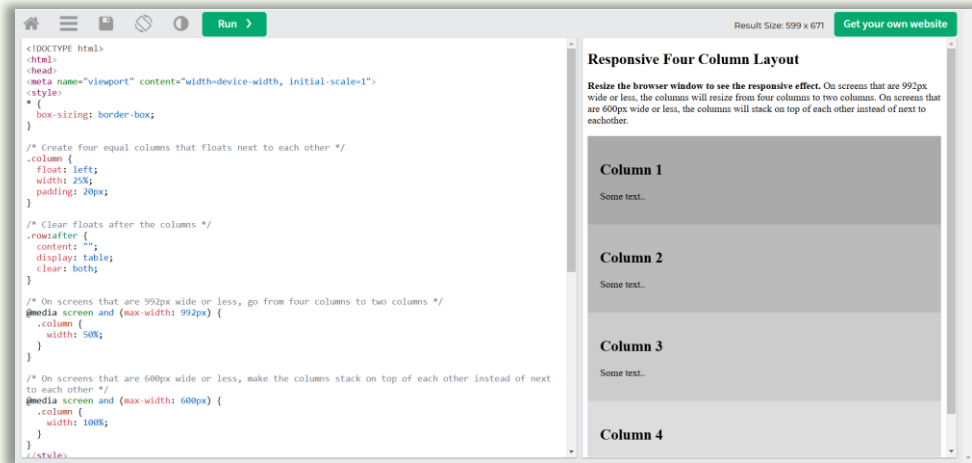
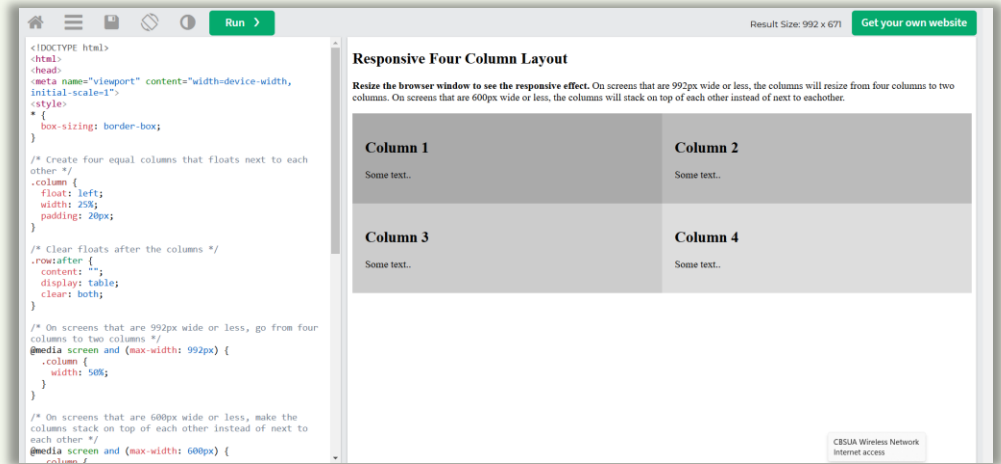
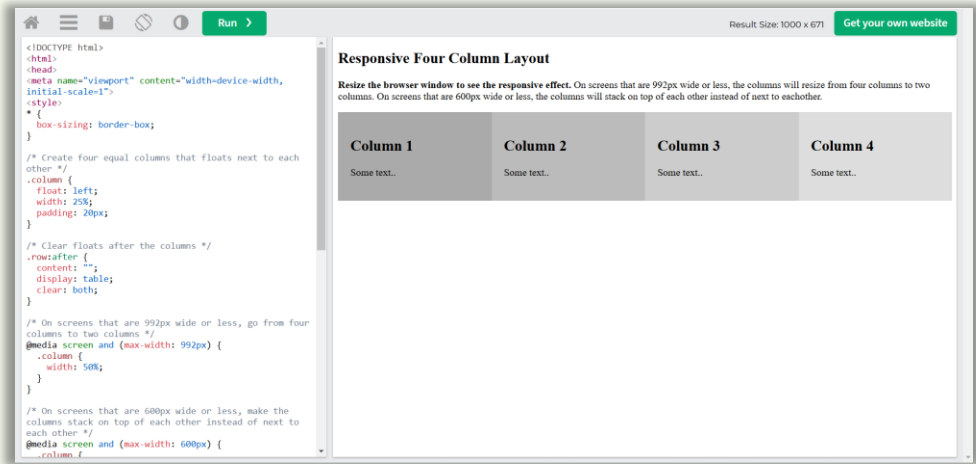
```
/* On screens that are 992px wide or less, go from four columns to two columns */
@media screen and (max-width: 992px) {
  .column {
    width: 50%;
  }
}

/* On screens that are 600px wide or less, make the columns stack on top of each other instead of next to each other */
@media screen and (max-width: 600px) {
  .column {
    width: 100%;
  }
}
```


Media Queries

Example 3

Output:



References

- <https://developer.mozilla.org>
- <https://web.dev/learn/css/welcome>
- https://www.w3schools.com/css/css_box_model.asp
- <https://developer.mozilla.org/en-US/docs/Web/CSS>
- <https://web.dev/learn/design/media-queries>
- https://www.w3schools.com/cssref/atrule_media.php
- https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Values_and_units
- <https://flexboxfroggy.com/>